University of Nottingham
UK | CHINA | MALAYSIA

COMP2001/2011: Artificial Intelligence Methods

Attendance Monitoring
QR Code ⬇️

Lecture 7 – Hyper-heuristics 1

Dr Warren G Jackson

The landscape so far…

- Previously we have looked at:
  - Components of heuristic search
  - Simple “low-level” heuristics (perturbative/hill-climbing)
  - Single-point and Population based Metaheuristics

- Today’s Topics:
  - Raising the level of generality even further: hyper-heuristics
    - Selection hyper-heuristics
    - HyFlex/Example Domain/CHeSC 2011/Cross-domain search
    - Heuristic selection methods
    - Examples of selection hyper-heuristics (MSHH)
    - Parameter tuning for cross-domain search

Heuristics

- Heuristics are problem specific methods used to define the neighbourhood of a solution.
  - Specific to certain representations.
  - Can be highly specialised to certain problems.
  - High development cost and time/effort.

Metaheuristics

- Metaheuristics (MH's) provide high-level problem-independent guidelines for developing heuristic optimisation algorithms.
- However, MHs require problem specific details to be designed/implemented/configured/tuned.
- Require domain expertise to design the algorithmic components.
- State-of-the-art methods require a trial-and-error process:
  - Design and implement algorithmic components.
  - Configure the algorithm and tune the parameters.
  - Analyse the performance and perhaps retune or reconfigure.

Higher-level Solvers?

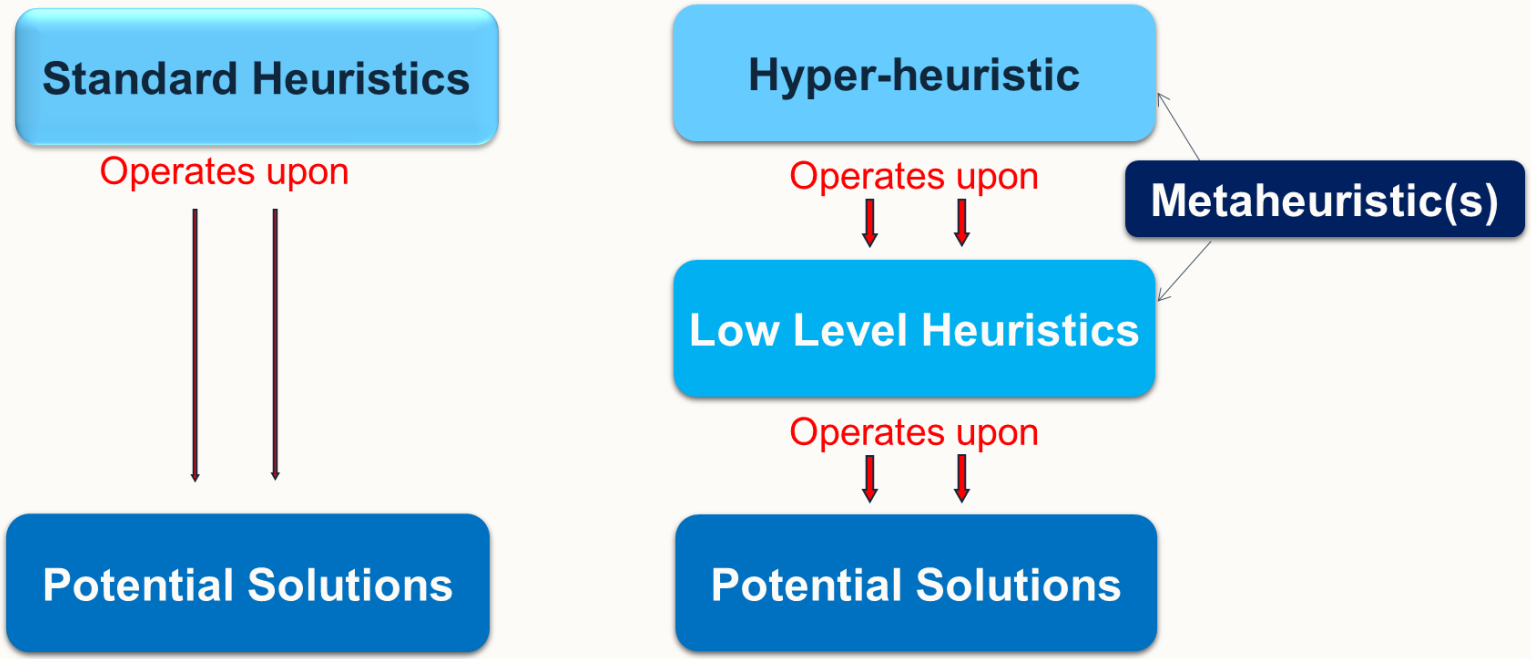
- What if we could design a **high-level solver** which:
  - Does not require problem specific details.
  - Is low-cost (requires no maintenance after the design phase).
  - Can be used for solving different problem domains.
  - May be good for solving a few different problems?
- Can we design a **general solver** which:
  - Does not require problem specific details.
  - Does not require tuning/re-configuring for solving different problem domains.
  - Has a high performance across different problem domains.

Hyper-heuristics

In search of higher-level solvers.

Hyper-heuristics

- ***Hyper-heuristics*** *(HH's) are high-level search methods or learning mechanisms for selecting or generating heuristics to solve computationally difficult problems.*
- Key difference is that HH's operate upon the search space of low-level heuristics whereas MH's operate upon the search space of solutions.

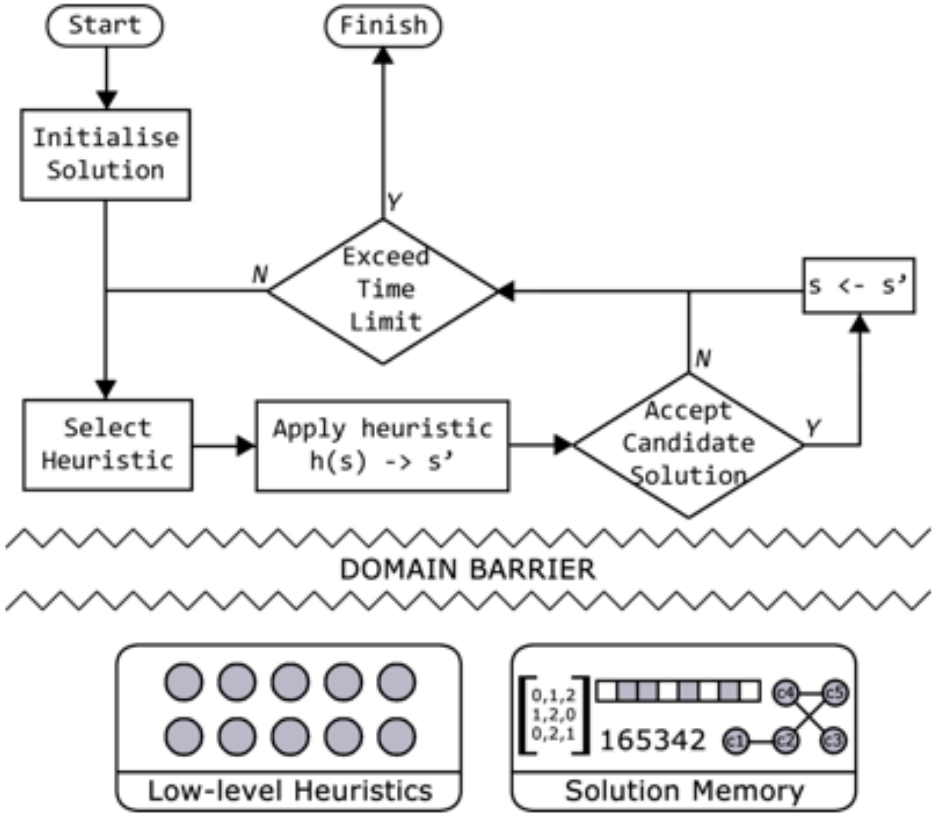


Why Hyper-heuristics?

- Hyper-heuristics research is motivated by raising the level of generality of search methods – aim towards the general solver.
- Existing methods are highly specialised and have good performance for solving a specific problem, however:
  - Do not perform well for solving other problems.
  - May not even be able to be applied for solving other problems.
- Hyper-heuristics aim to bridge this gap.

An Initial Selection Hyper-heuristic Framework

- HHs coined in the 2001 paper though dates back as far as the 1960s.
- HHs introduce a layer of abstraction known as the problem domain barrier.
- No problem specific knowledge is allowed to pass from the domain to the hyper-heuristic.
- The hyper-heuristic is only able to invoke heuristics from a predefined set of generic knowledge poor low-level heuristics.

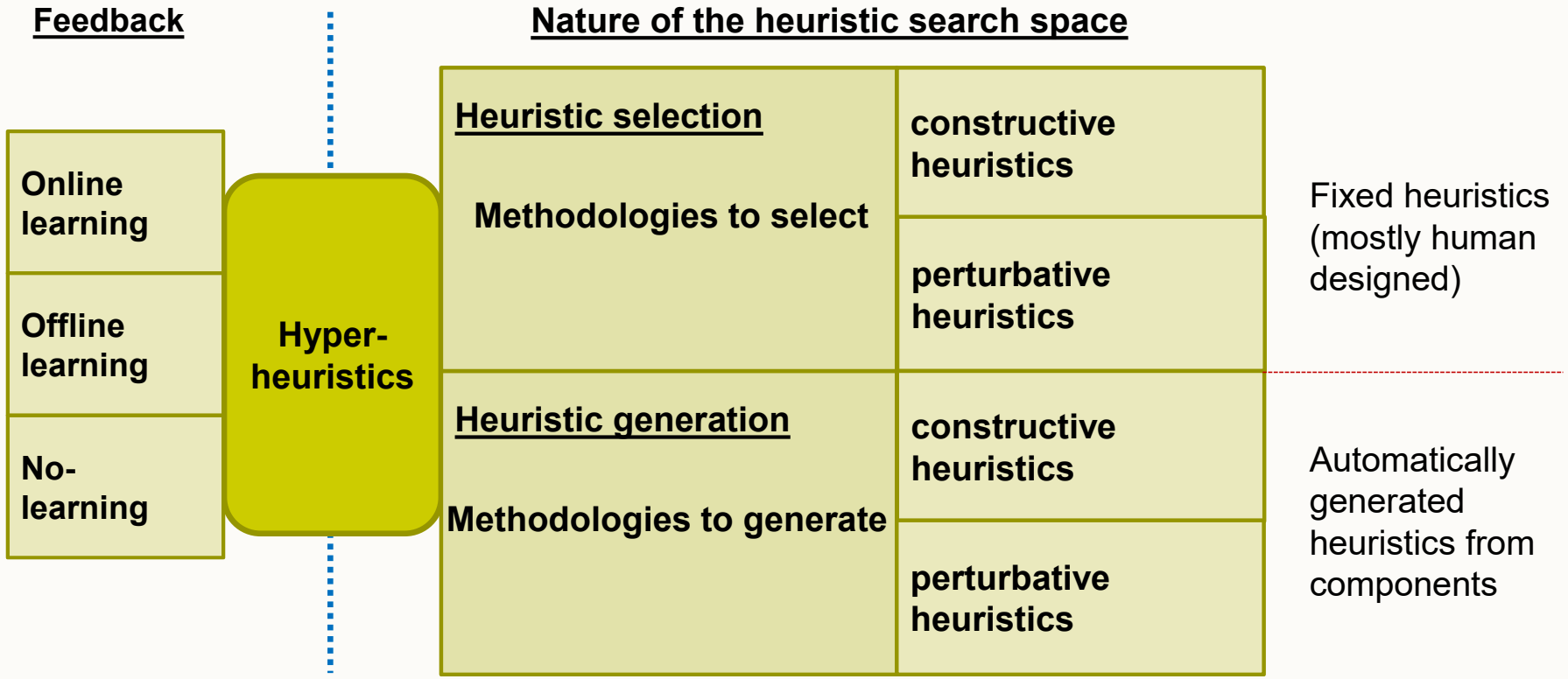


Cowling P.I., Kendall G. and Soubeiga E., 2001. A Hyperheuristic Approach to Scheduling a Sales Summit, selected papers from PATAT 2000, Springer, LNCS 2079, 176-190. [Link to the paper].

Characteristics of the Initial Hyper-heuristic Framework

- HHs operate on a **search space of heuristics** as opposed to methods seen so far that operate on the solutions themselves.
- Aim is for HHs to take advantage of the strengths of the low-level heuristics and avoid their weaknesses.
- No problem specific knowledge is used during the search over the heuristic space.
- Easy to implement, deploy, and use.
- Existing heuristics/metaheuristics can be used within hyper-heuristics.

A Classification of Hyper-heuristics



E. K. Burke, M. Hyde, G. Kendall, G. Ochoa, E. Özcan and J. Woodward, A Classification of Hyper-heuristic Approaches: Revisited, In Gendreau, M, and Potvin, JY. (eds.), Handbook of Metaheuristics, International Series in Operations Research & Management Science, vol. 272, pp. 453-477. Springer Cham, 2019. [Link to the paper]

Frameworks for Heuristic Selection/Generation

- SATzilla: algorithm portfolio-oriented data-driven framework
- Simple Neighborhood-based Algorithm Portfolio in PYthon (snappy)
- **Hyper-heuristics Flexible Interface (HyFlex): https://people.cs.nott.ac.uk/pszwj1/chesc2011/**
- ParHyFlex extends MPI: https://tinyurl.com/zrfz2vw
- Hyperion provides a white-box framework giving a metaheuristic/hyper-heuristic full access to the problem domain: https://github.com/MitLware/mitl-hyperion
- ECJ: http://cs.gmu.edu/~eclab/projects/ecj/

The Hyper-heuristics Flexible Interface (HyFlex) Framework

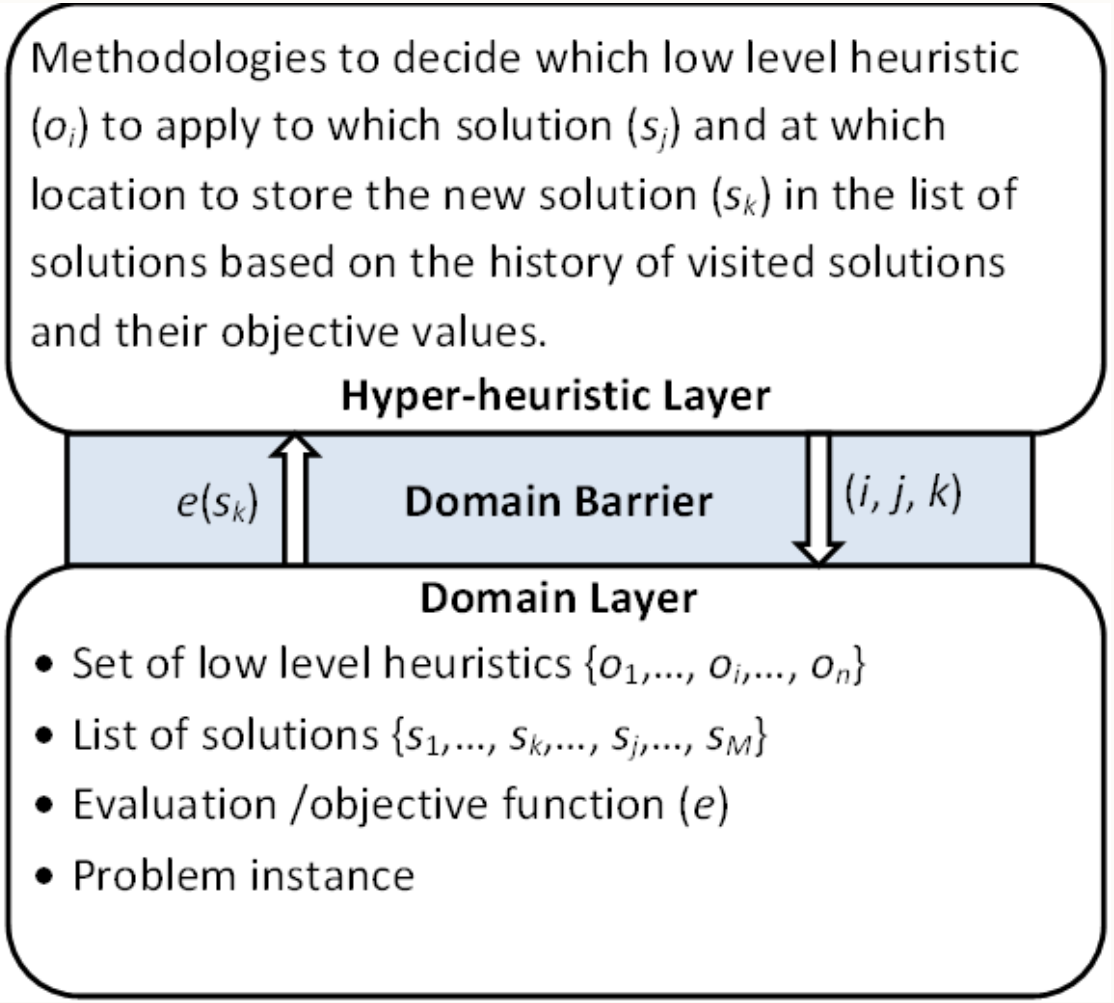
A Hyper-heuristics Framework for Selection Hyper-heuristics

Selection Hyper-heuristic Template

```
1 | generate initial solution s_0
2 | set current solution s as s_0
3 | WHILE termination_criteria_not_satisfied:
      // heuristic selection
4 |   select heuristic(s) h ∈ H+ to apply
5 |   generate new solution s' by applying h to s
      // move acceptance
6 |   decide to accept or reject s'
7 |   IF accept s' THEN s ← s'
8 | END_WHILE
9 | return s_best
```

Hyper-heuristics Flexible Interface (HyFlex)

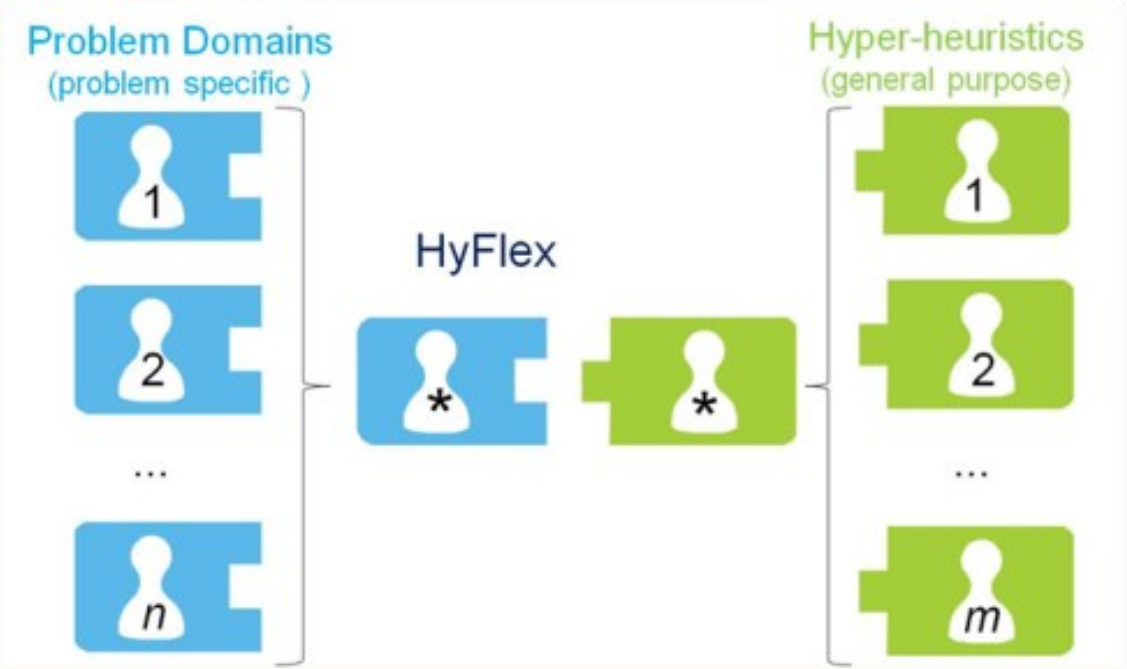
- HyFlex was designed as a framework to allow the development and testing of perturbative selection hyper-heuristics as part of a competition (CHeSC: Cross-Domain Heuristic Search Competition).
- Domain barrier restricts the flow of information between the hyper-heuristic layer (top) and domain layer (bottom).



Ochoa G, Hyde M, Curtois T, Vazquez-Rodriguez JA, Walker J, Gendreau M, Kendall G, McCollum B, Parkes AJ, Petrovic S, Burke EK (2012) HyFlex: a benchmark framework for cross-domain heuristic search. In: Evolutionary Computation in Combinatorial Optimization, LNCS 7245, pp 136–147. [Link to the paper].

HyFlex Framework (1)

- The HyFlex framework facilitates an abstraction between problem specific details, and the general purpose HHs.
- This separation allows for the reuse of components.
  - Any problem can be solved by any hyper-heuristic without modification.



HyFlex Framework (2)

- Provides six problem domain implementations.
- Each domain has a set of low-level heuristics.
- Heuristics may be parameterised:
  - Depth of search
  - Intensity of mutation
- HH can control these settings and select low-level heuristics based on their types.
  - Mutation
  - Ruin-recreate
  - Local search
  - Crossover
- Solutions are always complete.

MAX-SAT	Bin Packing	Flow Shop
Personnel Scheduling	TSP	VRP

Heuristic IDs	LLH0	LLH1	LLH2	LLH3	LLH4	LLH5	LLH6	LLH7
MAX-SAT	MU_0	MU_1	MU_2	MU_3	MU_4	**MU_5**	**RC_0**	HC_0
Bin Packing	MU_0	RC_0	**RC_1**	MU_1	HC_0	**MU_2**	**HC_1**	**XO_0**
PS	HC_0	HC_1	HC_2	HC_3	**HC_4**	RC_0	RC_1	**RC_2**
PFS	MU_0	MU_1	MU_2	MU_3	**MU_4**	RC_0	**RC_1**	HC_0
TSP	MU_0	MU_1	MU_2	MU_3	**MU_4**	**RC_0**	HC_0	HC_1
VRP	MU_0	MU_1	RC_0	**RC_1**	HC_0	XO_0	**XO_1**	**MU_2**

Heuristic IDs	LLH8	LLH9	LLH10	LLH11	LLH12	LLH13	LLH14
MAX-SAT	**HC_1**	XO_0	**XO_1**				
PS	XO_0	XO_1	**XO_2**	**MU_0**			
PFS	HC_1	HC_2	**HC_3**	XO_0	XO_1	XO_2	**XO_3**
TSP	**HC_2**	XO_0	XO_1	XO_2	**XO_3**		
VRP	HC_1	**HC_2**					

Example Domain Components: 1-Dimensional Bin Packing – Objective Function

- **Bin Packing (1D):** given a set of items with sizes $s_i \in \mathbb{Z}\,[1, C]$ for $i = 1, \ldots, n$ where C is a fixed capacity of each bin, pack each item into a set of bins to minimise the number of bins used while never exceeding the capacity of any bin.
- **Objective function** minimises emptiness of bins (maximises bin fullness) bin fullness across all open bins.
- *Note that all objective functions in HyFlex are assumed to be minimisation.*

$$f(x) = 1 - \left(\frac{\sum_{i=1}^{n} (fullness_i / C)^2}{n} \right)$$

such that

$$fullness_i = \sum_{j=1}^{n_{bi}} w_{b_{i,j}}$$

where

n denotes the number of bins,

C denotes the bin capacity,

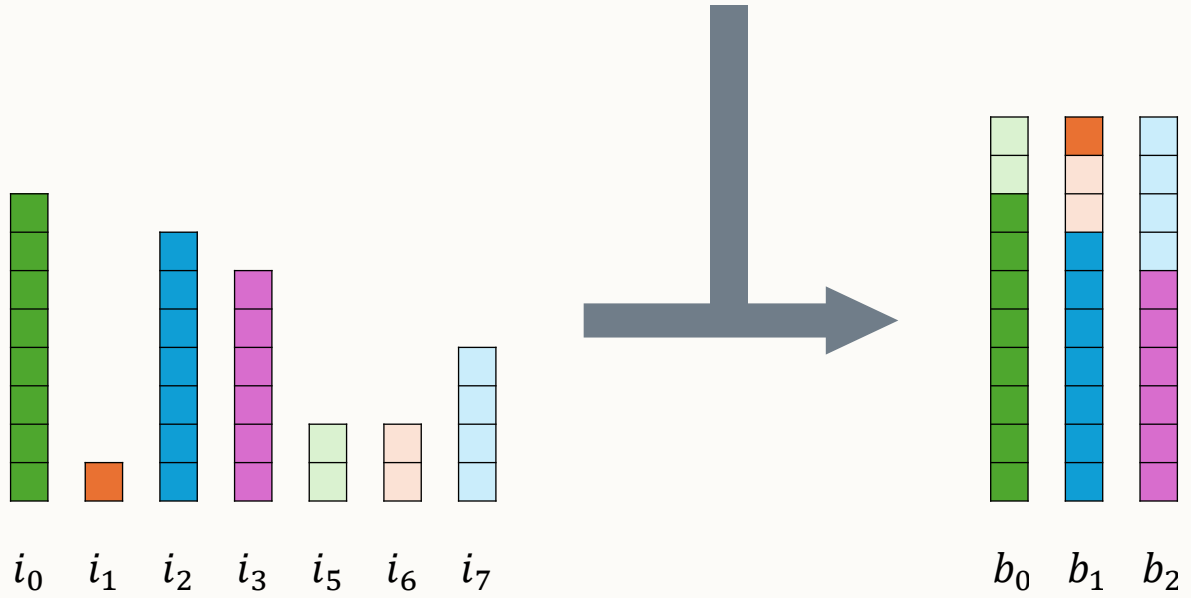
$b_{i,j}$ denotes the j^{th} piece in bin i,

n_{bi} denotes the number of pieces in bin b_i, and

$w_{b_{i,j}}$ is the weight of the j^{th} piece in bin i.

Example Domain Components: 1-Dimensional Bin Packing – Instances

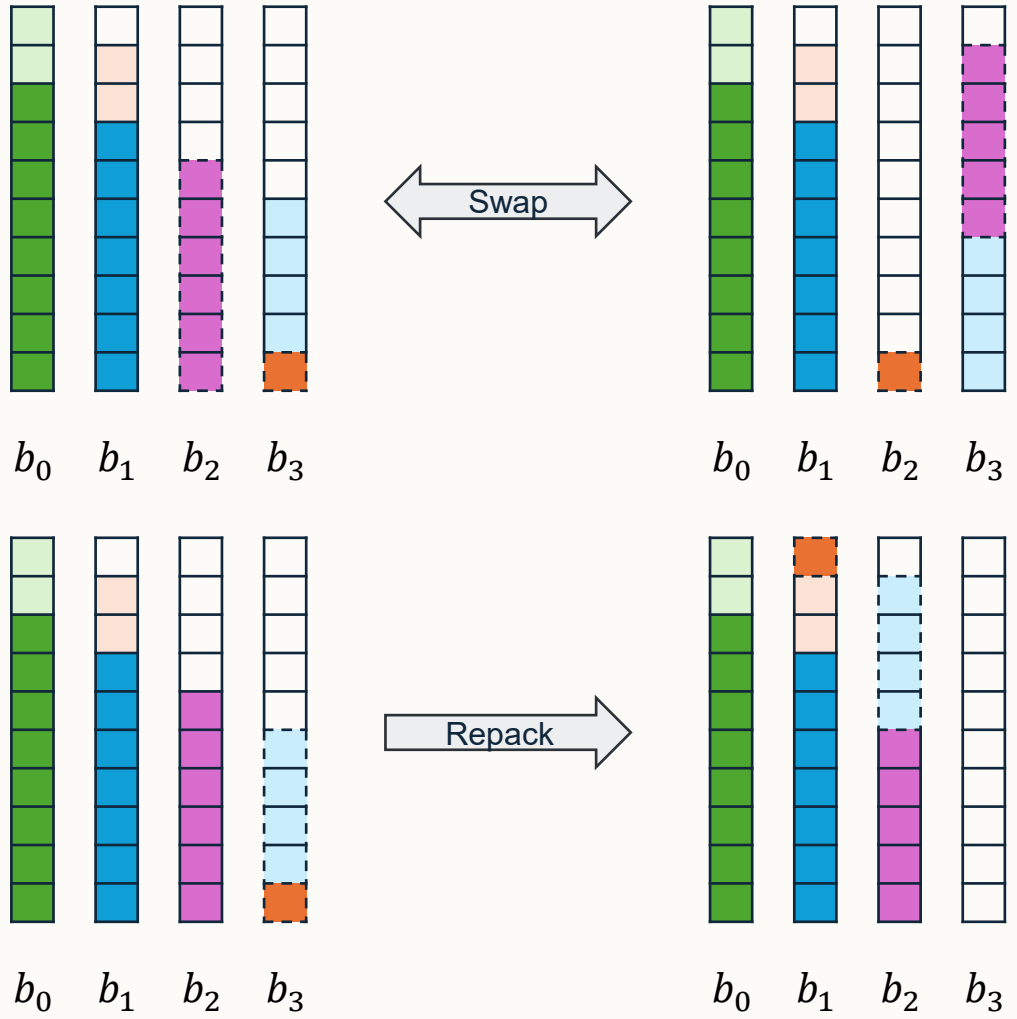
- **Example instance:** given a set of (n=7) items with sizes $S = \{8, 1, 7, 6, 2, 2, 4\}$ and a capacity C=10.
- **Solution representation**: < 1, 2, 2, 3, 1, 2, 3 >



Example Domain Components: 1-D Bin Packing – Mutation Heuristics

- **MU_0 (Swap)** – Selects two random items at random and swaps their bins if there is space.
- **MU_1 (Split a bin)** – Selects bin at random from the set of bins with more items than the average bin and splits it into two bins containing half the number of items each.
- **MU_2 (Repack lowest filled bin)** – removes all items from the lowest filled bin and tries to repack them into the remaining bins with the best-fit heuristic.

- **Intensity of mutation** defines how many times each LLH is repeated with 0.2 = repeat once through 1.0 = repeat 5 times.



Example Domain Components: 1-D Bin Packing – Hill Climbing Heuristics

- **HC_0 (First Improvement (IE) with Swap)** – Swaps two items at random if there is space and keeping the new solution if it results in a non-worsening solution cost.
- **HC_1 (First Improvement (IE) with Swap from lowest filled bin)** – Takes the largest item from the lowest filled bin and swaps it with the smallest item from a randomly selected bin (subject to feasibility and improvement).

- **Depth of search** defines how many times each LLH is repeated with 0.2 = repeat 10 times through 1.0 = repeat 20 times.

Example Domain Components: 1-D Bin Packing – Ruin-&-Recreate Heuristics

- **RC_0 (Destroy *x* highest bins)** – Remove all pieces from the *x* highest filled bins and repack using best fit.
- **RC_1 (Destroy *x* lowest bins)** – Remove all pieces from the *x* lowest filled bins and repack using best fit.
- **Intensity of mutation** defines how many times each LLH is repeated with 0.2 = repeat 3 times through 1.0 = repeat 15 times.
- **Crossover XO_0 (Exon Shuffling Crossover) – see:**
  - Philipp Rohlfshagen and John Bullinaria. A genetic algorithm with exon shuffling crossover for hard bin packing problems. In Proceedings of the 9th annual conference on Genetic and evolutionary computation (GECCO'07), pages 1365–1371, London, U.K., 2007.

The Cross-domain Heuristic Search Challenge 2011

CHeSC 2011 - CHeSC: Cross-Domain Heuristic Search Competition

CHeSC 2011 Competition

- An international competition to try and find the general solver!
- Used the HyFlex Framework to evaluate selection hyper-heuristics.

MAX-SAT

Bin Packing

Flow Shop

Personnel Scheduling

TSP

VRP

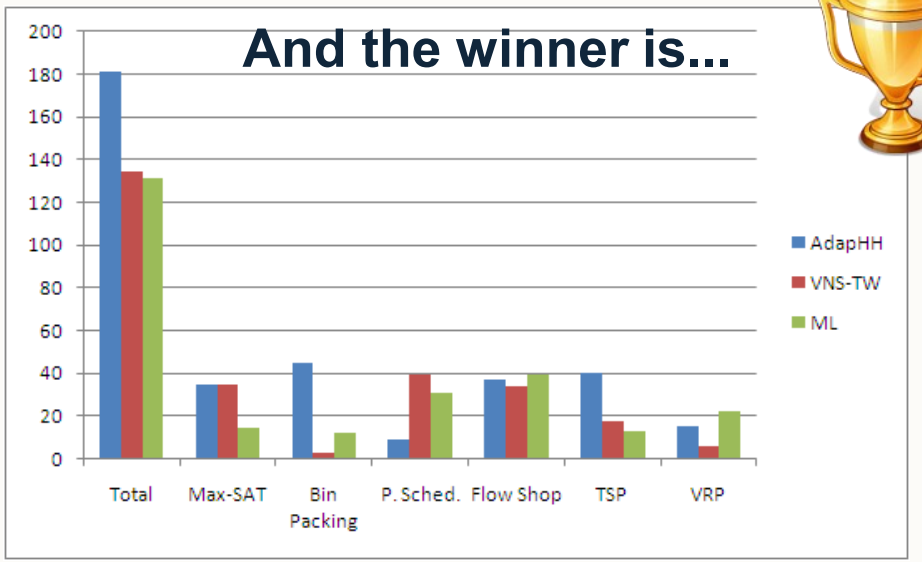
Hidden

- 10 public training instances per domain
- **5 instances for evaluation:**
  (3 from training + 2 hidden/all hidden)

- Set problem instance
- Set time limit (10 minutes)
- Perform 31 runs
- Report median

Ranking: Formula 1 scoring system

Rank	Score
1	10
2	8
3	6
4	5
5	4
6	3
7	2
8	1
rest	0



M. Misir, K. Verbeeck, P. D. Causmaecker, and G. V. Berghe, “An intelligent hyper-heuristic framework for chesc 2011,” in Proceedings of Learning and Intelligent Optimization (LION 2012), ser. LNCS, Y. Hamadi and M. Schoenauer, Eds., vol. 7219. Paris, France: Springer, 2012, pp. 461–466.

CHeSC 2011 Competition Results

- Running all competitors on all 30 instances for 31 trials each found AdapHH to be the best hyper-heuristic for cross-domain search.

Rank	Hyper-heuristic	Score	Rank	Hyper-heuristic	Score
1	AdapHH	181.00	11	ACO-HH	39.00
2	VNS-TW	134.00	12	GenHive	36.50
3	ML	131.50	13	DynILS	27.00
4	PHUNTER	93.25	14	SA-ILS	24.25
5	EPH	89.75	15	XCJ	22.50
6	HAHA	75.75	16	AVEG-Nep	21.00
7	NAHH	75.00	17	GISS	16.75
8	ISEA	71.00	18	SelfSearch	7.00
9	KSATS-HH	66.50	19	MCHH-S	4.75
10	HAEA	53.50	20	Ant-Q	0.00

Selection Hyper-heuristics

Heuristic Selection and Move Acceptance

Selection Hyper-heuristics

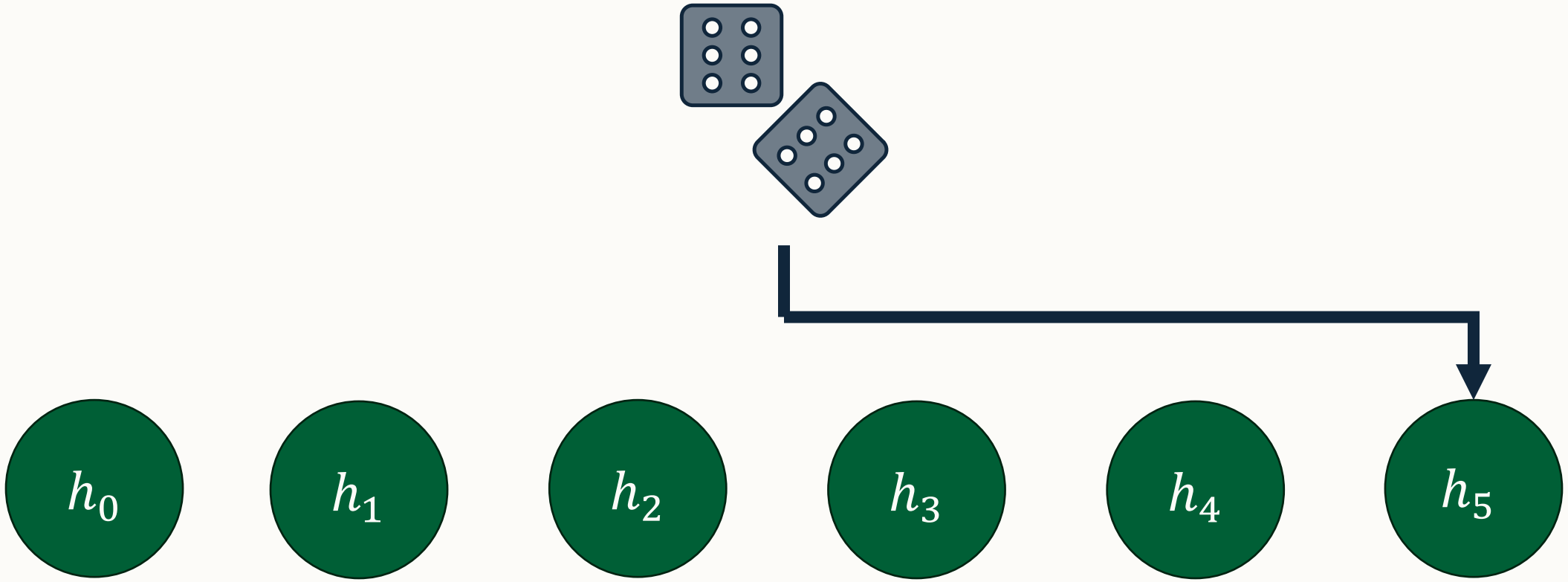
- A class of hyper-heuristics that **select** from one or more pre-defined low-level heuristics to be applied to the solution-in-hand at a given point in the search.
- **Non-learning selection hyper-heuristics** may choose from a set of low-level heuristics arbitrarily.
- **Selection hyper-heuristics** contain learning mechanisms for selecting the best heuristic from a set of heuristics at any given point during the search.

Heuristic Selection Methods – Further Reading

Category	Component name	Reference(s)
Heuristic selection with no learning	Simple Random	Cowling et al (2000, 2002b)
Heuristic selection with no learning	Random Permutation	Cowling et al (2000, 2002b)
Heuristic selection with learning	Peckish	Cowling and Chakhlevitch (2003)
Heuristic selection with learning	Greedy	Cowling et al (2000, 2002b); Cowling and Chakhlevitch (2003)
Heuristic selection with learning	Random Gradient	Cowling et al (2000, 2002b)
Heuristic selection with learning	Random Permutation Gradient	Cowling et al (2000, 2002b)
Heuristic selection with learning	Choice Function	Drake et al (2015)
Heuristic selection with learning	Reinforcement Learning	Nareyek (2003); Pisinger and Ropke (2007)
Heuristic selection with learning	Reinforcement Learning with Tabu Search	Burke et al (2003); Dowsland et al (2007)
Heuristic selection with learning	Quality Index and Tabu-based Learning Heuristic Selection	Misir et al (2009, 2012)
Heuristic selection with learning	Dominance-based Selection	Kheiri and Özcan (2011; 2015)
Heuristic selection with learning	Probability-based Selection	Lehrbaum and Musliu (2012)
Heuristic selection with learning	Adaptive pursuit	Walker et al (2012)

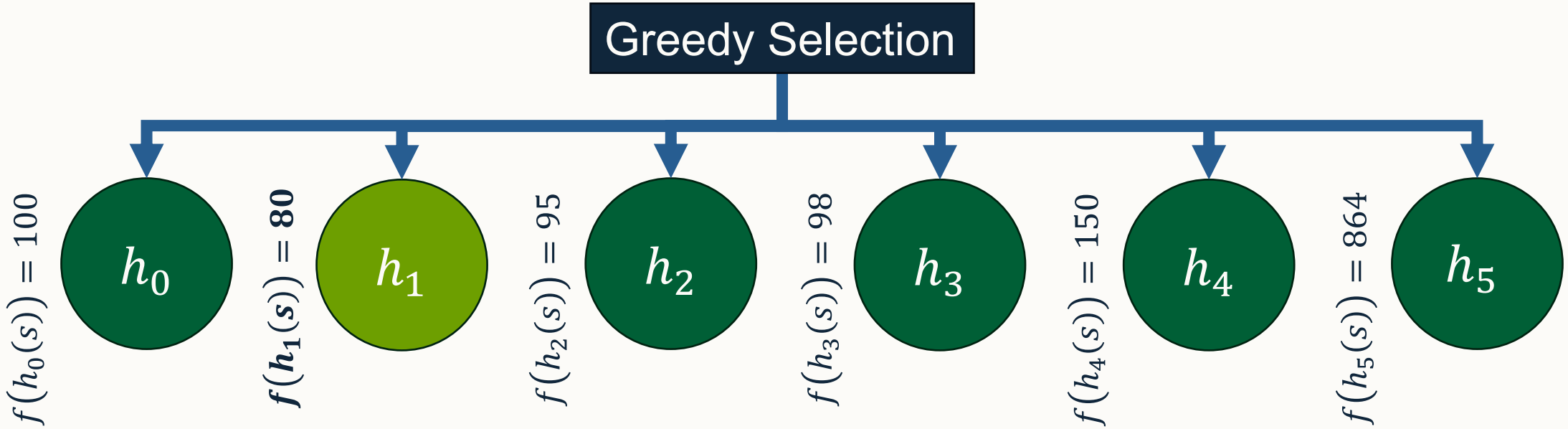
Non-learning Selection Methods (1)

- **Random selection** applies a low-level heuristic at random to the solution in hand.



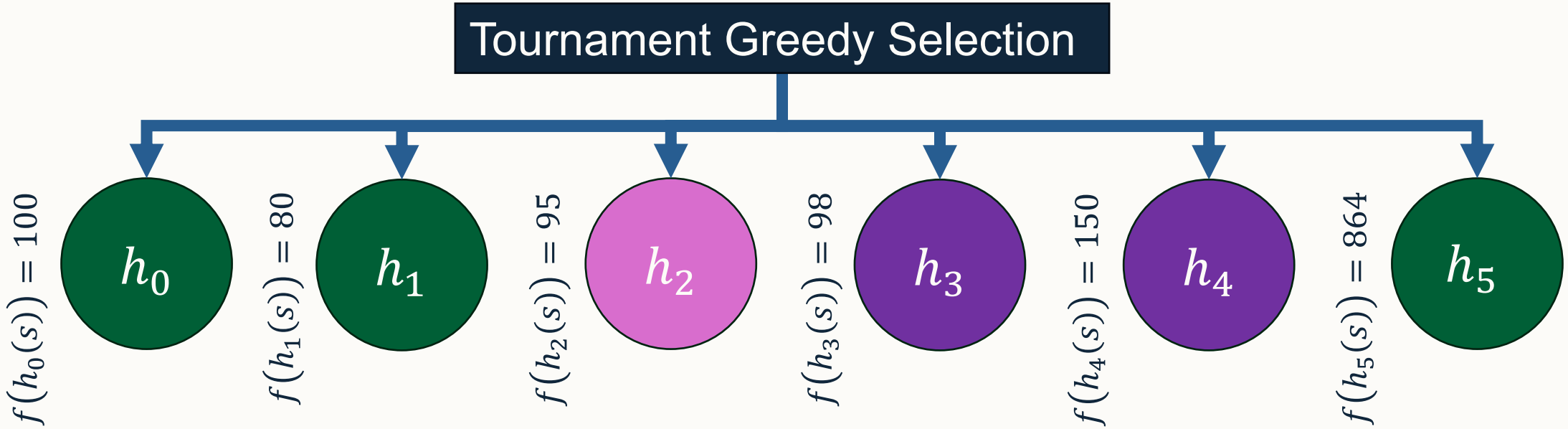
Non-learning Selection Methods (2)

- **Greedy selection** applies each low-level heuristic in turn and chooses the one that generates the solution with the best objective value.



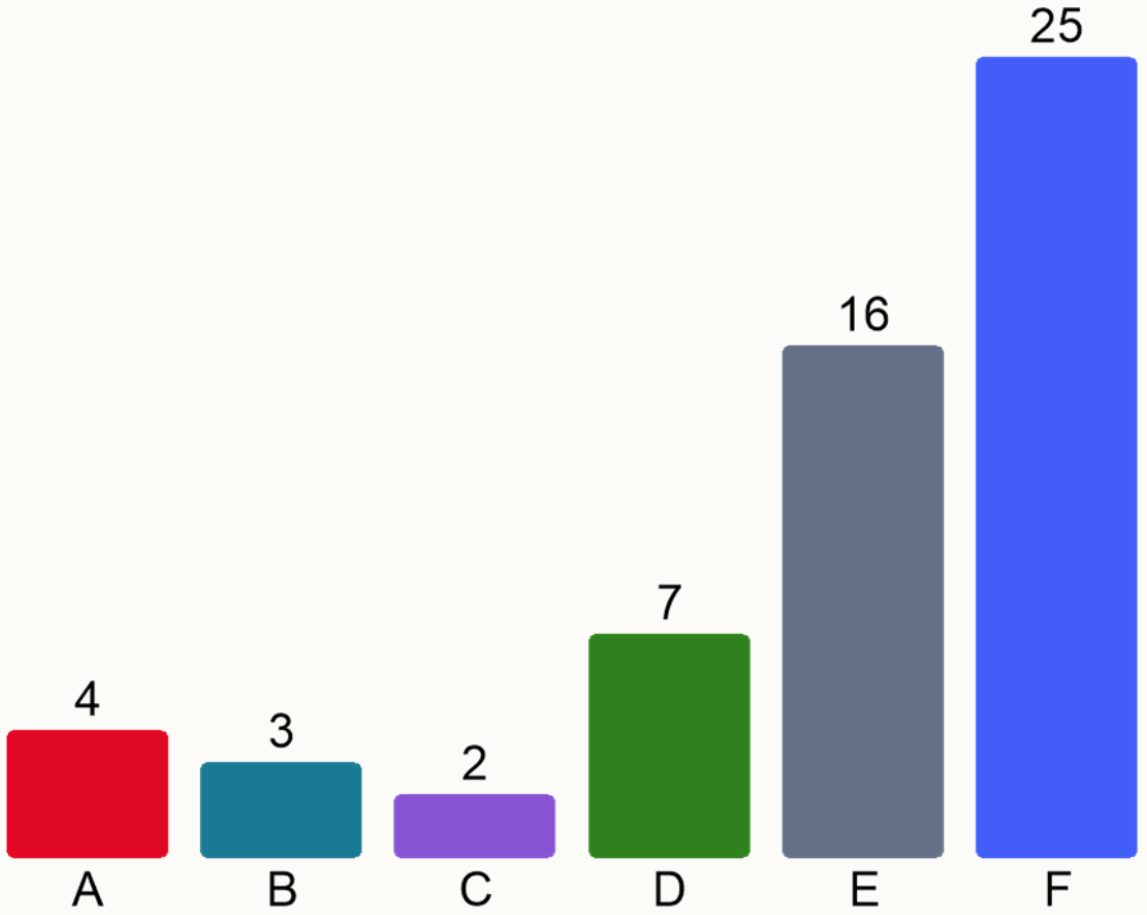
Non-learning Selection Methods (3)

- **Tournament greedy selection** applies a subset of t randomly chosen low-level heuristics and chooses the one that generates the solution with the best objective value.



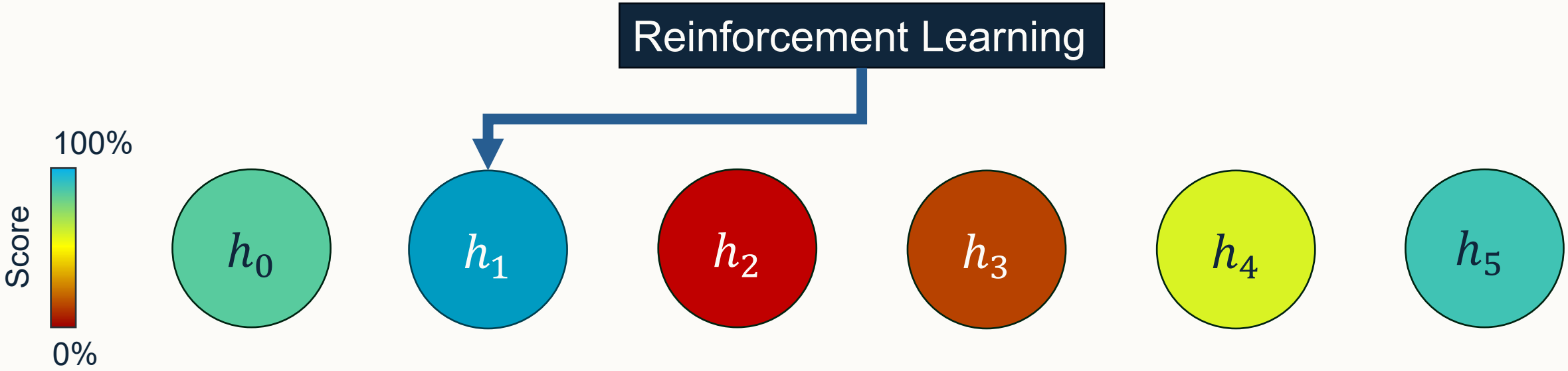
Which of the following heuristics can never be selected using tournament greedy selection with t = 3 for the following search state on the slides?

A. h_0 with $f(h_0(s)) = 50$
B. h_1 with $f(h_1(s)) = 55$
C. h_2 with $f(h_2(s)) = 58$
D. h_3 with $f(h_3(s)) = 60$
✓E. h_4 with $f(h_4(s)) = 62$
✓F. h_5 with $f(h_5(s)) = 70$



Learning-based Selection Methods: Reinforcement Learning

- A simple machine learning technique inspired by “reward and punishment” from psychology.
- Reward those heuristics that are “good” and punish those that are “bad”.



Reinforcement Learning: General Template

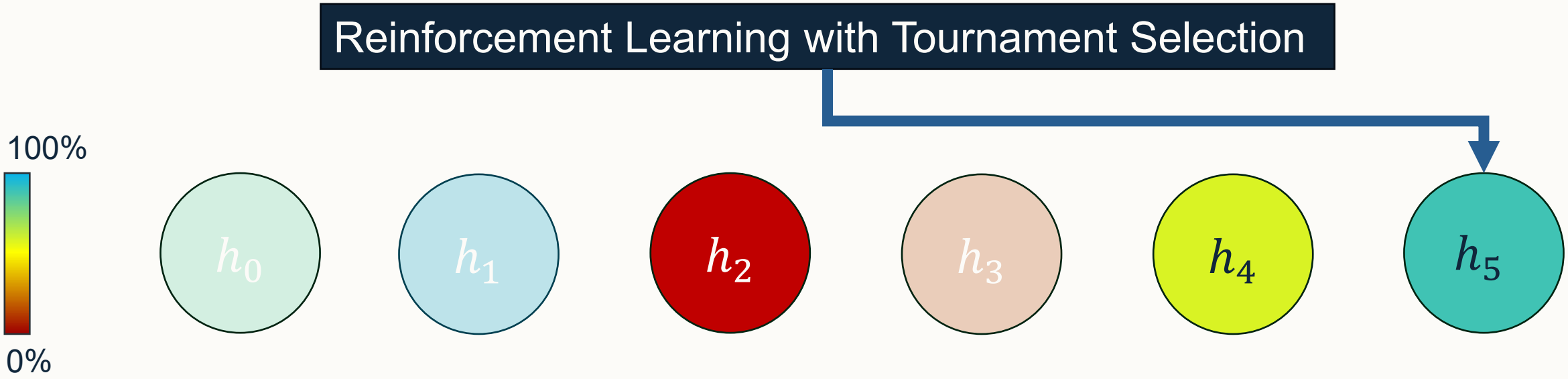
```
0 | INPUTS: s, heuristics, scores, …
1 | h ← selectHeuristic(heuristics, scores)
2 | s’ ← h(s)
3 | IF f(s’) isBetterThan f(s) THEN
4 |   scores[h].reward()
5 | ELSE
6 |   scores[h].punish()
```

Reinforcement Learning: Reward and Punishment

- Can reward/punish LLHs by incrementing/decrementing scores.
- May also chose to update scores:
  - Multiplication by some factor.
  - Asymmetric – reward and punishment have different values.
- Should scores have bounds?
- What to do if `f(s') == f(s)`?
- Initialisation of scores?
- What to do in cases of tied scores?

Reinforcement Learning: Selection 1

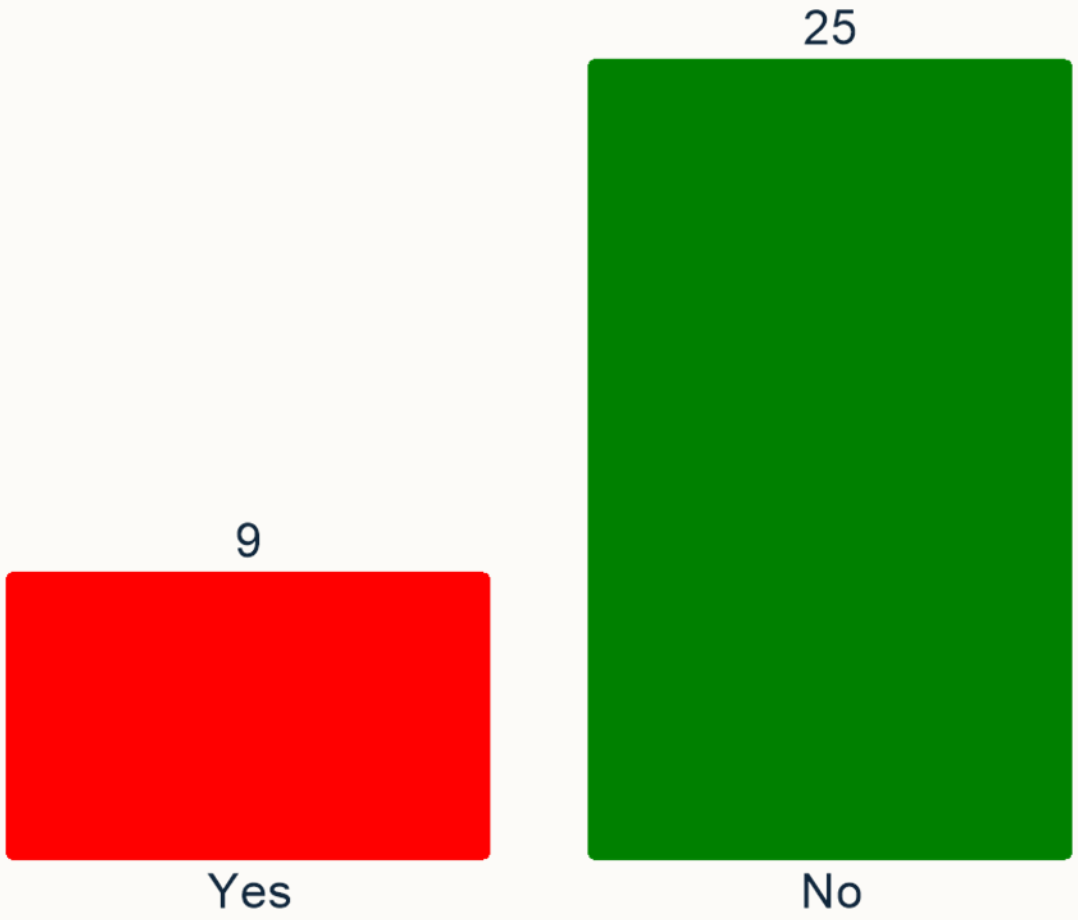
- Initial example we saw the selection of the best scoring LLH.
- Tournament selection (like in EAs) chose the best from *tournament_size* randomly selected LLHs.



If reinforcement learning is used with a tournament size = 1, is this still a learning-based selection hyper-heuristic?

A. Yes

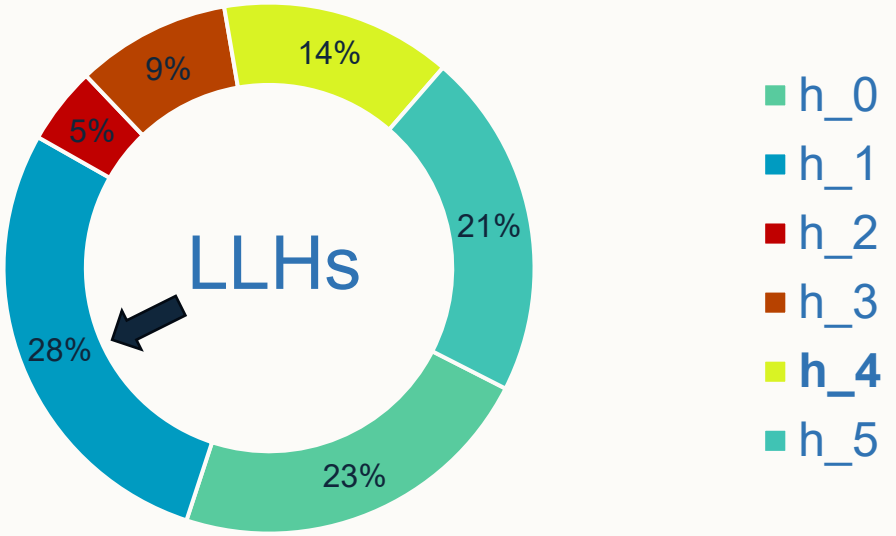
✓B. No



Reinforcement Learning: Selection 2

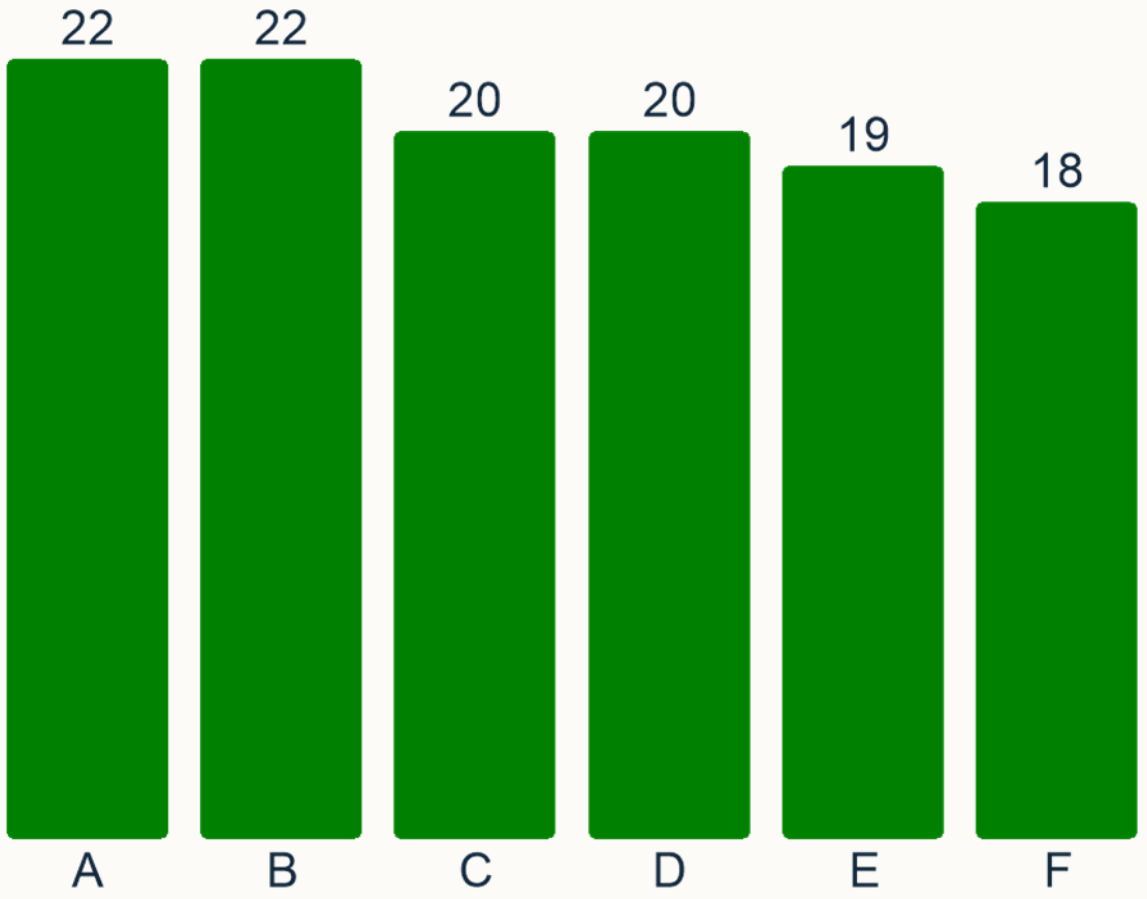
- Can also randomly chose a LLH randomly based on probability scoring.
- (Roulette Wheel/Fitness Proportionate) Selection selects randomly but the odds of a LLH being chosen scales proportionately to their score.

- Probability to select h_x is calculated as $P(h_x) = \frac{scores[h_x]}{\sum_{i=0}^{scores.length-1} scores[h_i]}$



Which of the following heuristics could be selected by Roulette Wheel Selection given the associated proportions?

- ✓A. h_0 with $P(h_1) = 50\%$
- ✓B. h_1 with $P(h_1) = 20\%$
- ✓C. h_2 with $P(h_2) = 15\%$
- ✓D. h_3 with $P(h_3) = 9\%$
- ✓E. h_4 with $P(h_4) = 5.999\%$
- ✓F. h_5 with $P(h_5) = 0.001\%$



Learning-based Selection Methods: Choice Function

- Considers multiple performance metrics of a LLH to assign a score:
  1. Individual performance.
  2. How well the LLH performed when applied after the previously applied LLH.
  3. The time since the LLH was last applied – concept of “age”.

- $F(h_j) = \alpha f_1(h_j) + \beta f_2(h_k, h_j) + \delta f_3(h_j)$

- Choice Function Hyper-heuristic chooses the LLH with the best score.
- In theory can use any of the other mechanisms we saw previously.

Components of Choice Function: f_1

- The “individual performance” metric.
- $f_1(h_j) = \sum_n \alpha^{n-1} \frac{I_n(h_j)}{T_n(h_j)}$ where
  - $I_n(h_j)$ is the change in solution quality,
  - $T_n(h_j)$ is the time taken to run the heuristic,
  - n is the number of times h_j has been applied, and
  - $\alpha \in [0,1]$

Components of Choice Function: f_2

- How well the LLH performed when applied after the previously applied LLH.
- $f_2(h_k, h_j) = \sum_n \beta^{n-1} \frac{I_n(h_k,h_j)}{T_n(h_k,h_j)}$ where
  - $I_n(h_k, h_j)$ is the change in solution quality*,
  - $T_n(h_k, h_j)$ is the time taken to run the heuristic*,
  - n is the number of times h_j has been applied*, and
  - $\beta \in [0,1]$

- *when applying h_j after h_k.

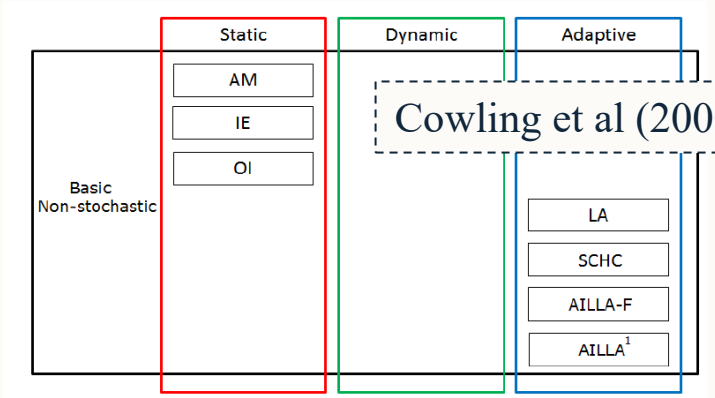
Components of Choice Function: f_3

- The age of a heuristic since last used.
- $f_3(h_j) = \tau(h_j)$ where
  - $\tau(h_j)$ is the time since h_j was last applied.

Move Acceptance

- While selection hyper-heuristics aim to choose the best low-level to apply, they generally still need to be guided by a move acceptance method.
  - Low-level heuristics may be stochastic in nature.
  - A heuristic that was previously “good” may no longer be that good now.
  - Still need to guide the search along an intelligent trajectory.

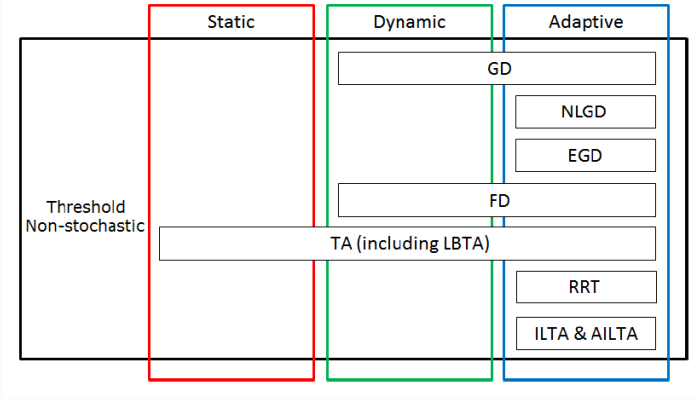
Move Acceptance in Selection HH's



Cowling et al (2000, 2002b)

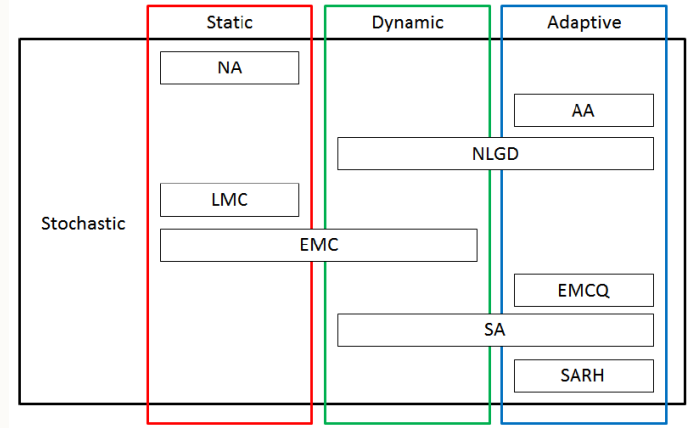
Özcan et al (2009);
Jackson et al. (2013)

Mısır et al (2012)



Kendall and Mohamad (2004b) || Bilgin et al. (2006)

Kheiri and Özcan (2015)
Kendall and Mohamad (2004b)
Mısır et al (2009)



Burke et al (2010); Kheiri and Özcan (2012); Asta and Özcan (2015)

Ayob and Kendall (2003)

Bai and Kendall (2005); Bilgin et al (2006); Pisinger and Ropke (2007); Antunes et al (2009)

Dowsland et al (2007); Bai et al. (2007a)

Selection Hyper-heuristic Framework for Single-Point Search

```
1 | generate initial solution s_0
2 | set current solution s as s_0
3 | WHILE termination_criteria_not_satisfied:
      // heuristic selection
4 |   select heuristic(s) h ∈ H+ to apply
5 |   generate new solution s' by applying h to s
      // move acceptance
6 |   decide to accept or reject s'
7 |   IF accept s' THEN s ← s'
8 | END_WHILE
9 | return s_best
```

Common Misconceptions

- Hyper-heuristics do not require parameter tuning.
- Hyper-heuristics are always tested under a fair comparison.
  - Training instances vs test instances.
  - Computational budget needs to be set per machine.
    - Benchmark designed for older architectures.
  - Timer resolution across operating systems – affects learning processes.
    - 1/64 second accuracy on Windows
  - Timer drift on Windows 11.
- Applying a hyper-heuristic to a new problem domain is easy.
  - Easy to run but first need to implement the new domain and components while respecting any API. (E.g. HyFlex).

Focus of recent research on HHs

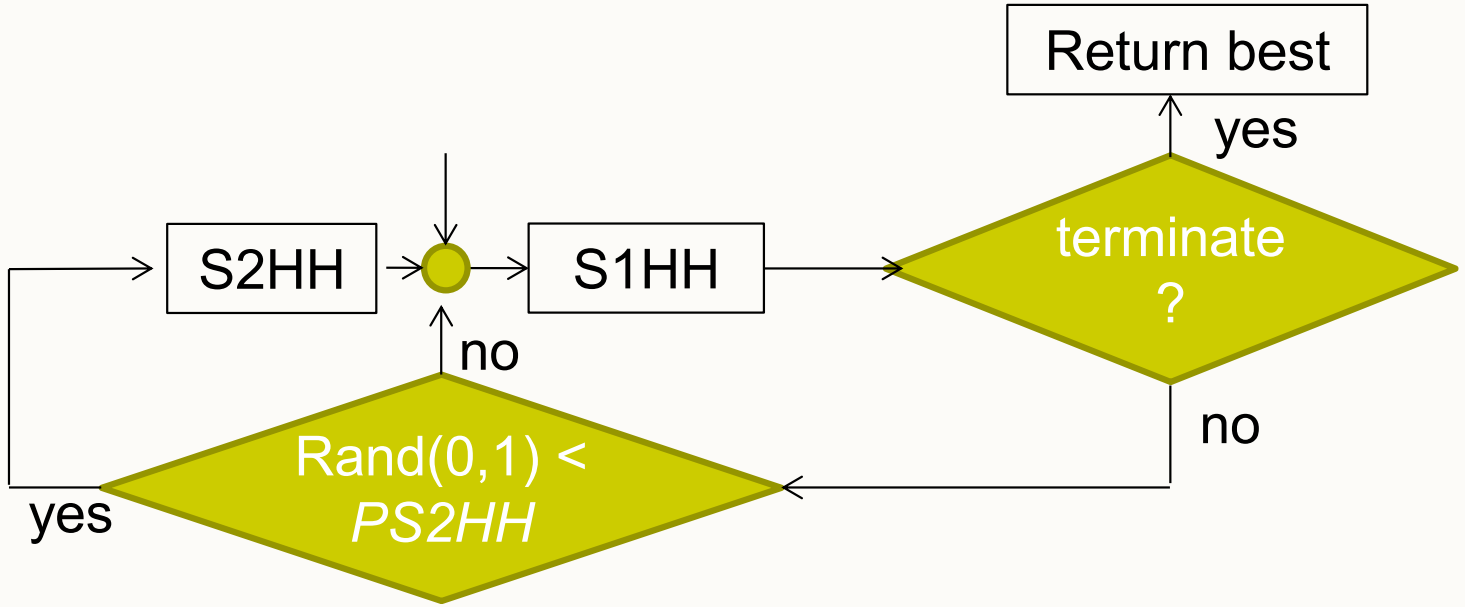
- **Existing hyper-heuristics are tuned/configured for cross-domain performance improvement.**
- Existing hyper-heuristics applied to new domains.
- Design of new hyper-heuristics:
  - Hyper^2-heuristics
  - **Multi-stage approaches**
  - Generation of hyper-heuristics
  - Multi-objective hyper-heuristics
- Hybrid-approaches mixing (exact/inexact) and (constructive/perturbative) methods.

An Iterated Multi-Stage Selection Hyper-heuristic (MSHH)

A. Kheiri and E. Özcan, An Iterated Multi-stage Selection Hyper-heuristic, European Journal of Operational Research, (250)1:77–90, 2016.

Basic Idea of MSHH

- There are two hyper-heuristics, S1HH and S2HH.
  - S1HH is fast but simple.
  - S2HH is slow but more sophisticated than S1HH.
  - S2HH is applied after S1HH based on a random probability.



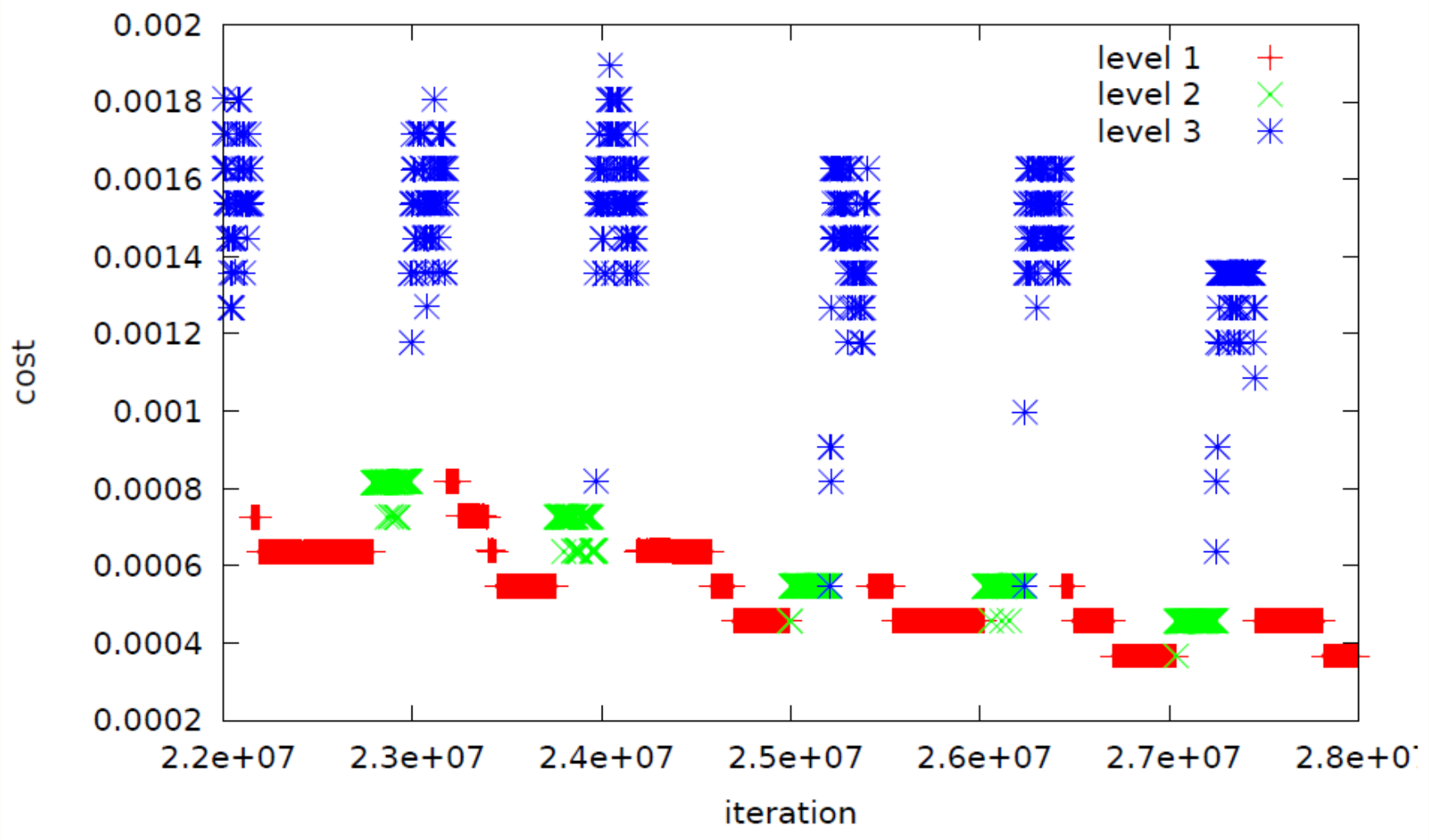
S1HH Overview

- Uses reinforcement learning to score heuristics and selects the LLH to apply using Roulette Wheel Selection – probabilities come from S2HH.
- Moves are accepted based on an adaptive threshold move acceptance method. (Next slide).
- S1HH terminates if a duration is exceeded without any improvement in the best solution found (in this stage).

S1HH Move Acceptance Method

- Similar to record-to-record travel but uses the stage best solution value and ε is determined adaptively.
- $accept(s', s, h)$ **IF** $f(s')$ *isBetterThan* $f(s)$ **OR** $f(s')$ *isBetterThan* $(1 + \varepsilon) \cdot f\left(s_{stageBest}\right)$
- $\varepsilon = \frac{\lfloor \log f\left(s_{stageBest}\right) \rfloor + C[i]}{f\left(s_{stageBest}\right)}$
- C is a circular list where $\mathrm{C}[i] \in \{C[0], \ldots C[n-1]\}$ which is updated in stage 2 but fixed within stage 1.
- ε is updated to the next value in the list if no improvement is found within a certain duration d.

Progress plot of S1HH with |C| = 3



S2HH Overview

- Uses **relay-hybridisation** to create a further N^2 LLHs. In total considers all original LLHs **plus** the pairings of all LLHs to form a total of $N + N^2$ LLHs.

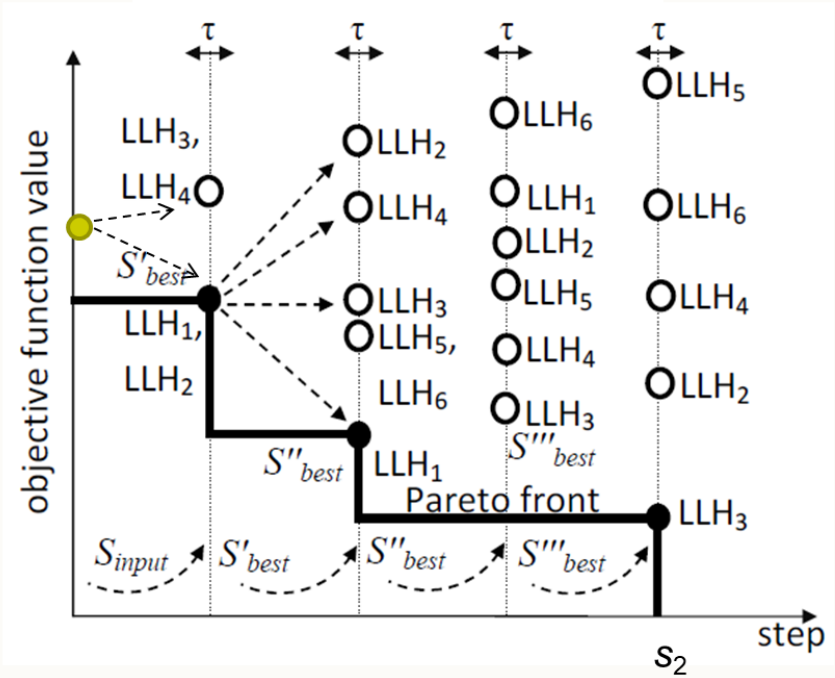
Given *N* LLHs, e.g., LLH_1, LLH_2

Pair up all and increase the number of LLHs to $N+N^2$

$LLH_3 \leftarrow LLH_1 + LLH_1$
$LLH_4 \leftarrow LLH_2 + LLH_2$
$LLH_5 \leftarrow LLH_1 + LLH_2$
$LLH_6 \leftarrow LLH_2 + LLH_1$

Reduce the Number of LLHs ($N+N^2 \rightarrow n$) + Assign Probabilities

LLH_1=2, LLH_2=1, LLH_3=1
50% 25% 25%



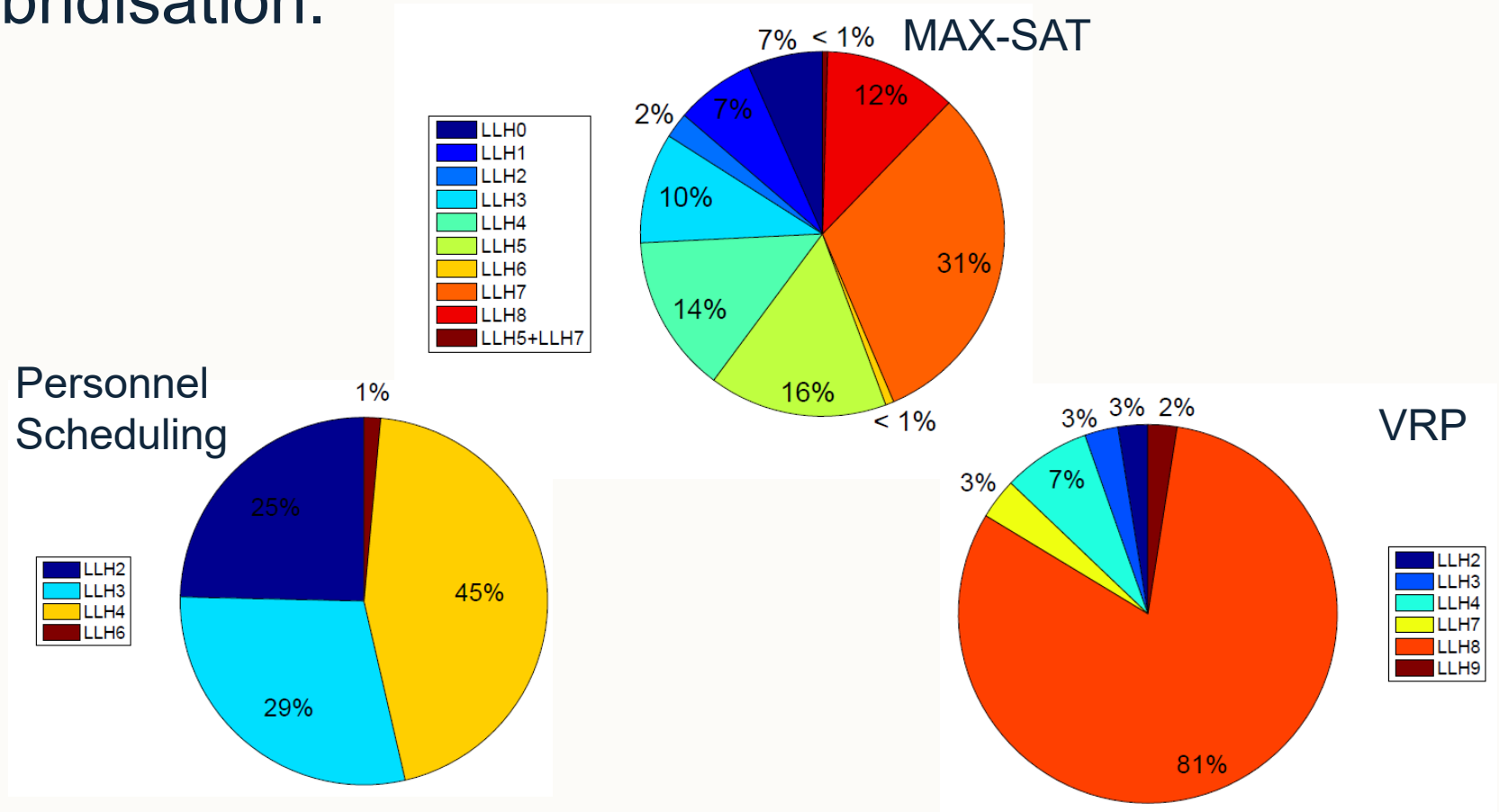
6 LLHs → **3 LLHs**

MSHH Contains Many Parameters!

- There are 6 parameters of the system and requires tuning.
  - $\tau = \{10, \mathbf{15}, 20, 30\}\ (ms)$
  - $d = \{7, \mathbf{9}, 10, 12\}\ (s)$
  - $s_1 = \{10, 15, \mathbf{20}, 25\}(s)$
  - $s_2 = \{3, \mathbf{5}, 10, 15\}(\text{steps/iterations})$
  - $P(S2HH) = \{0.1, \mathbf{0.3}, 0.6, 0.9, 1.0\}$
  - $C = \{\{0\}, \{3\}, \{6\}, \{9\}, \{\mathbf{0}, \mathbf{3}, \mathbf{6}, \mathbf{9}\}\}$

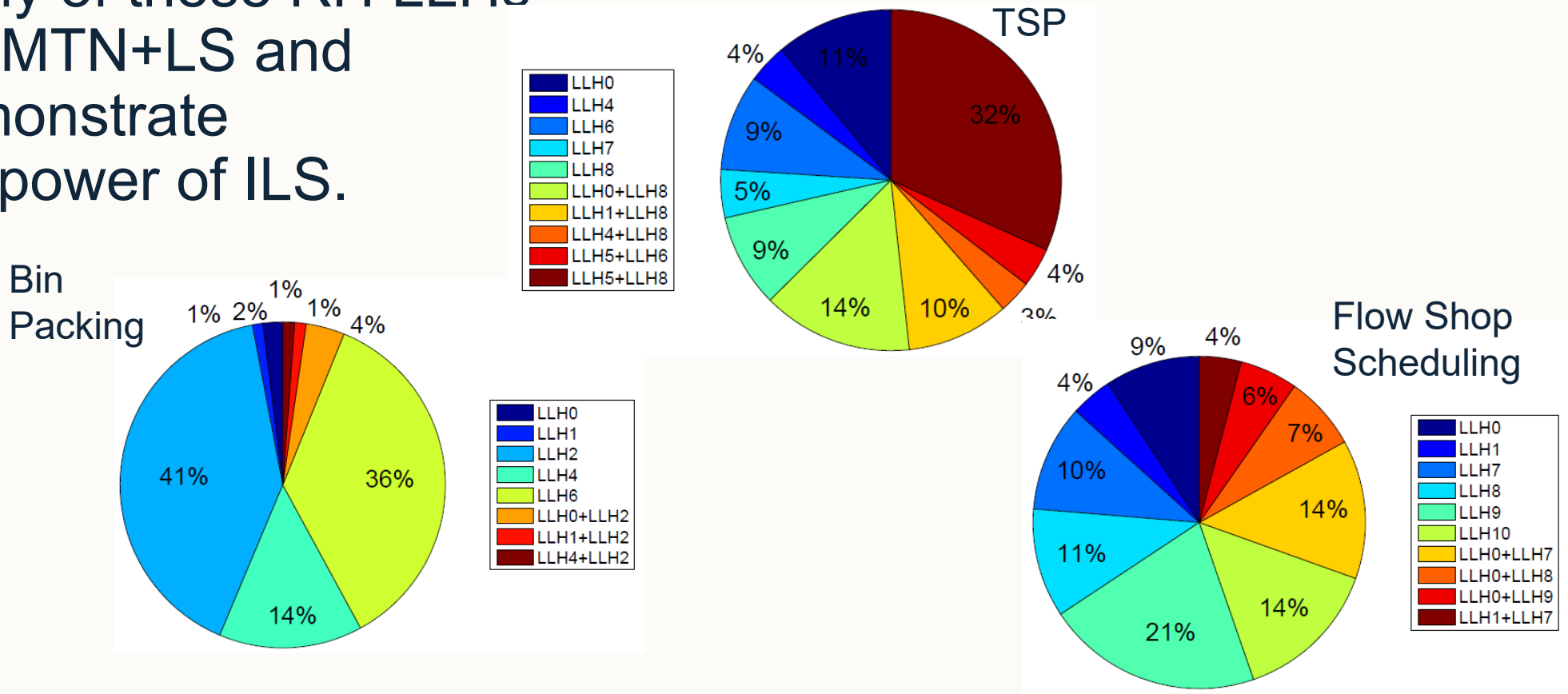
MSHH Utility of Relay Hybridisation (1)

- Some of the HyFlex/CHeSC domains did not benefit (at all/much) from Relay-Hybridisation.



MSHH Utility of Relay Hybridisation (2)

- While others greatly benefited from Relay-Hybridisation with some LLHs only being used alongside others.
- Many of these RH LLHs are MTN+LS and demonstrate the power of ILS.

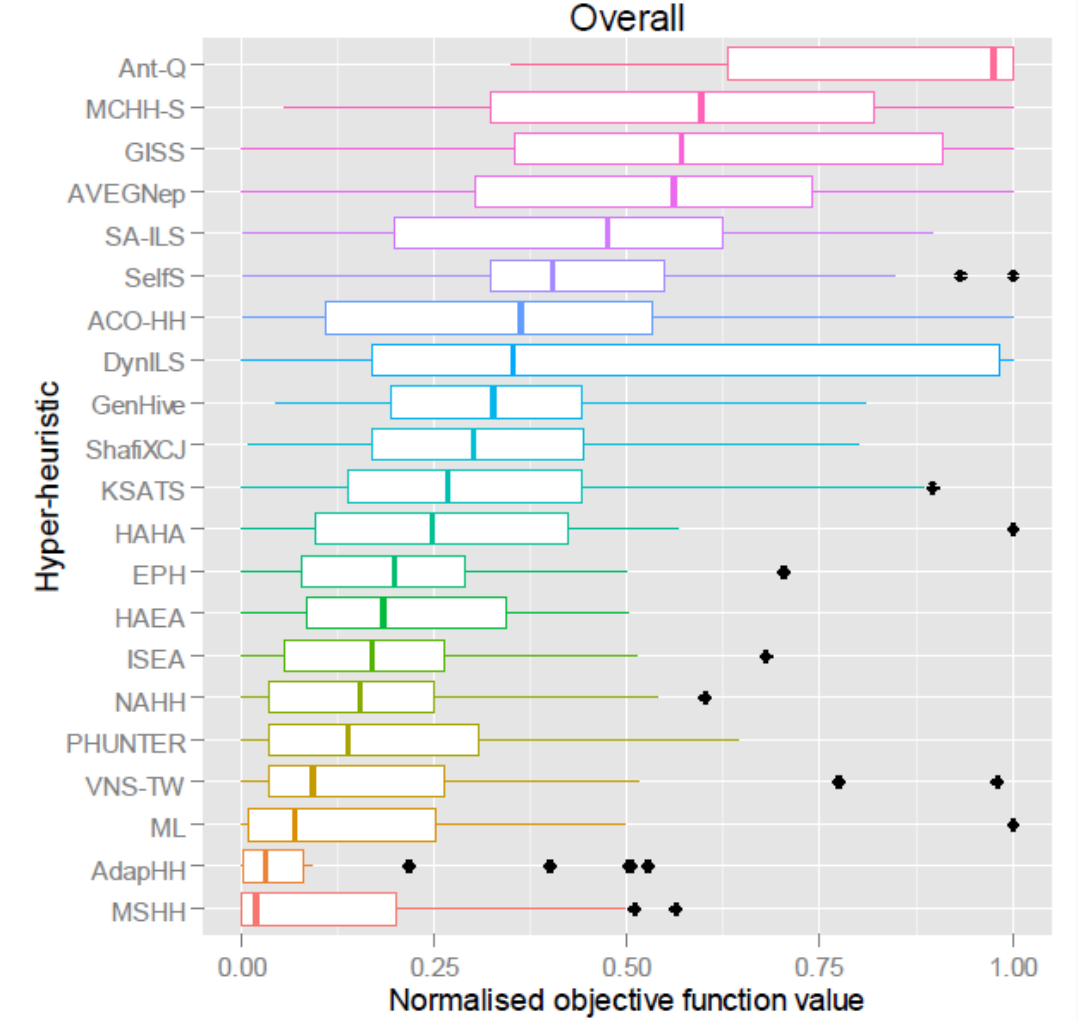


Performance Comparison of MS/S1/S2-HH

		MSHH					S1HH				S2HH		
Domain	Instance	avg.	std.	median	min.	vs.	avg.	std.	min.	vs.	avg.	std.	min.
SAT	Inst1	**0.9**	0.7	1.0	**0.0**	$>$	6.4	4.5	1.0	$>$	15.0	4.6	3.0
	Inst2	**3.1**	3.9	2.0	**1.0**	$>$	21.3	13.3	3.0	$>$	44.9	9.8	18.0
	Inst3	**0.7**	0.5	1.0	**0.0**	$>$	7.1	7.7	**0.0**	$>$	26.3	14.0	1.0
	Inst4	**1.7**	1.0	1.0	**1.0**	$>$	5.7	4.3	**1.0**	$>$	20.0	4.6	12.0
	Inst5	**7.6**	0.9	7.0	**7.0**	$>$	10.4	1.5	**7.0**	$>$	15.4	1.7	13.0
BP	Inst1	0.0163	0.0014	0.0163	**0.0136**	$<$	**0.0159**	0.0010	0.0137	$>$	0.0198	0.0015	0.0160
	Inst2	**0.0037**	0.0015	0.0030	**0.0025**	$>$	0.0061	0.0015	0.0034	$>$	0.0104	0.0021	0.0077
	Inst3	**0.0050**	0.0015	0.0049	**0.0025**	$\geq$	0.0054	0.0012	0.0027	$>$	0.0128	0.0011	0.0104
	Inst4	0.1084	0.0000	0.1084	**0.1083**	$\leq$	**0.1084**	0.0000	0.1083	$>$	0.1084	0.0000	0.1084
	Inst5	**0.0050**	0.0019	0.0044	**0.0032**	$\geq$	0.0055	0.0021	0.0032	$>$	0.0210	0.0015	0.0187
PS	Inst1	**25.5**	4.5	25.0	**16.0**	$>$	28.8	4.7	18.0	$>$	31.6	4.9	22.0
	Inst2	9668.9	217.8	9638.0	**9184.0**	$\leq$	**9645.3**	159.6	9334.0	$\leq$	9645.8	106.7	9391.0
	Inst3	**3283.7**	93.3	3270.0	**3132.0**	$\geq$	3304.8	99.6	3134.0	$\geq$	3309.9	110.2	3172.0
	Inst4	**1786.3**	172.1	1760.0	1545.0	$\geq$	1801.0	142.3	1570.0	$\geq$	1836.0	291.1	**1400.0**
	Inst5	**353.2**	21.2	350.0	**315.0**	$>$	724.4	657.3	320.0	$>$	810.7	621.5	360.0
PFS	Inst1	**6239.8**	14.9	6239.0	**6212.0**	$>$	6287.6	21.9	6249.0	$>$	6353.3	29.8	6301.0
	Inst2	26895.2	55.3	26889.0	**26775.0**	$<$	**26873.2**	30.7	26822.0	$>$	26976.9	54.7	26849.0
	Inst3	**6333.8**	19.0	6325.0	**6303.0**	$>$	6360.5	16.4	6323.0	$>$	6405.5	23.7	6369.0
	Inst4	**11363.8**	32.7	11359.0	**11320.0**	$>$	11429.9	43.8	11357.0	$>$	11529.3	35.9	11436.0
	Inst5	26711.9	47.0	26709.0	26630.0	$\leq$	**26693.1**	40.7	**26608.0**	$>$	26779.1	49.8	26702.0
TSP	Inst1	**48208.1**	31.8	48194.9	**48194.9**	$>$	50032.0	571.1	49263.1	$>$	50326.5	606.6	49221.6
	Inst2	**2.09e^{+7}**	9.05e^{+4}	2.09e^{+7}	**2.07e^{+7}**	$>$	2.14e^{+7}	1.12e^{+5}	2.12e^{+7}	$>$	2.13e^{+7}	1.05e^{+5}	2.11e^{+7}
	Inst3	**6809.1**	7.1	6808.8	**6796.6**	$>$	7012.5	30.4	6964.6	$>$	7040.2	31.3	6988.6
	Inst4	**66840.2**	276.5	66843.6	**66236.8**	$>$	68908.4	382.4	68159.9	$>$	70241.9	704.6	68791.0
	Inst5	**53011.4**	469.7	52910.2	**52341.3**	$>$	54411.1	595.1	53686.0	$>$	55814.8	946.4	53992.4
VRP	Inst1	70998.4	3840.3	70506.5	**63948.2**	$\leq$	**70223.0**	2960.2	64273.2	$>$	84103.9	7225.8	68958.3
	Inst2	**13421.8**	251.6	13359.6	**13303.9**	$\geq$	13658.0	471.4	13319.6	$>$	13695.8	473.9	13320.0
	Inst3	148498.2	1625.8	148436.2	145466.5	$\leq$	**148232.6**	1935.3	145426.5	$\geq$	149553.2	2377.8	**145362.7**
	Inst4	21016.4	488.2	20671.4	**20650.8**	$\leq$	**20991.3**	478.0	20653.5	$>$	21131.9	510.3	20657.5
	Inst5	**148813.7**	1272.5	149193.7	**146334.6**	$\geq$	148999.1	1217.1	146844.9	$>$	150282.6	1616.3	146666.9

MSHH versus CHeSC 2011 Results

- Top with a CHeSC 2011 score of **163.60**



Parameter Tuning for Cross-domain Search

A case study with a steady state memetic algorithm.

D. B. Gumus, E. Özcan and J. Atkin, An Investigation of Tuning a Memetic Algorithm for Cross-domain Search, Proc. of the 2016 IEEE Congress on Evolutionary Computation (CEC), pp. 135-142, 2016. [Link to the paper].

Parameter Tuning for Cross-domain Search?

- We know parameter tuning is effective to improve the performance of heuristic search methods for solving a particular problem, but…
- Is parameter tuning of a single algorithm still beneficial for cross-domain search?

Cross-domain Parameter Tuning of SSMA

- A previous study shows that a Steady State Memetic Algorithm (SSMA) performs better than its trans-generational variant across 6 problem domains of HyFlex.[1]
- This study performs parameter tuning using the **Taguchi DoE** method and investigates its performance across 9 problem domains with 5 instances from each domain.

Algorithm 1 : Pseudocode of Steady-state Memetic Algorithm

```
Create a population of popsize random individuals
Set the parameter values for Intensity of Mutation (IoM),
Depth of Search (DoS) and tournament size (toursize)
Apply a random local search method (hill climbing) to each individual
while (termination criterion is not satisfied)
  Parent1 ← Select-Parent(population, toursize)
  Parent2 ← Select-Parent(population, toursize)
  Child   ← ApplyCrossover (Rand(1, MAX_CROSSOVER), Parent1,
Parent2)
  Child   ← ApplyMutation (Rand(1, MAX_MUTATION), IoM, Child)
  Child   ← ApplyLocalSearch(Rand(1, MAX_LOCALSEARCH), DoS,
Child)
  WorstOf(population) ← Child
end while
```

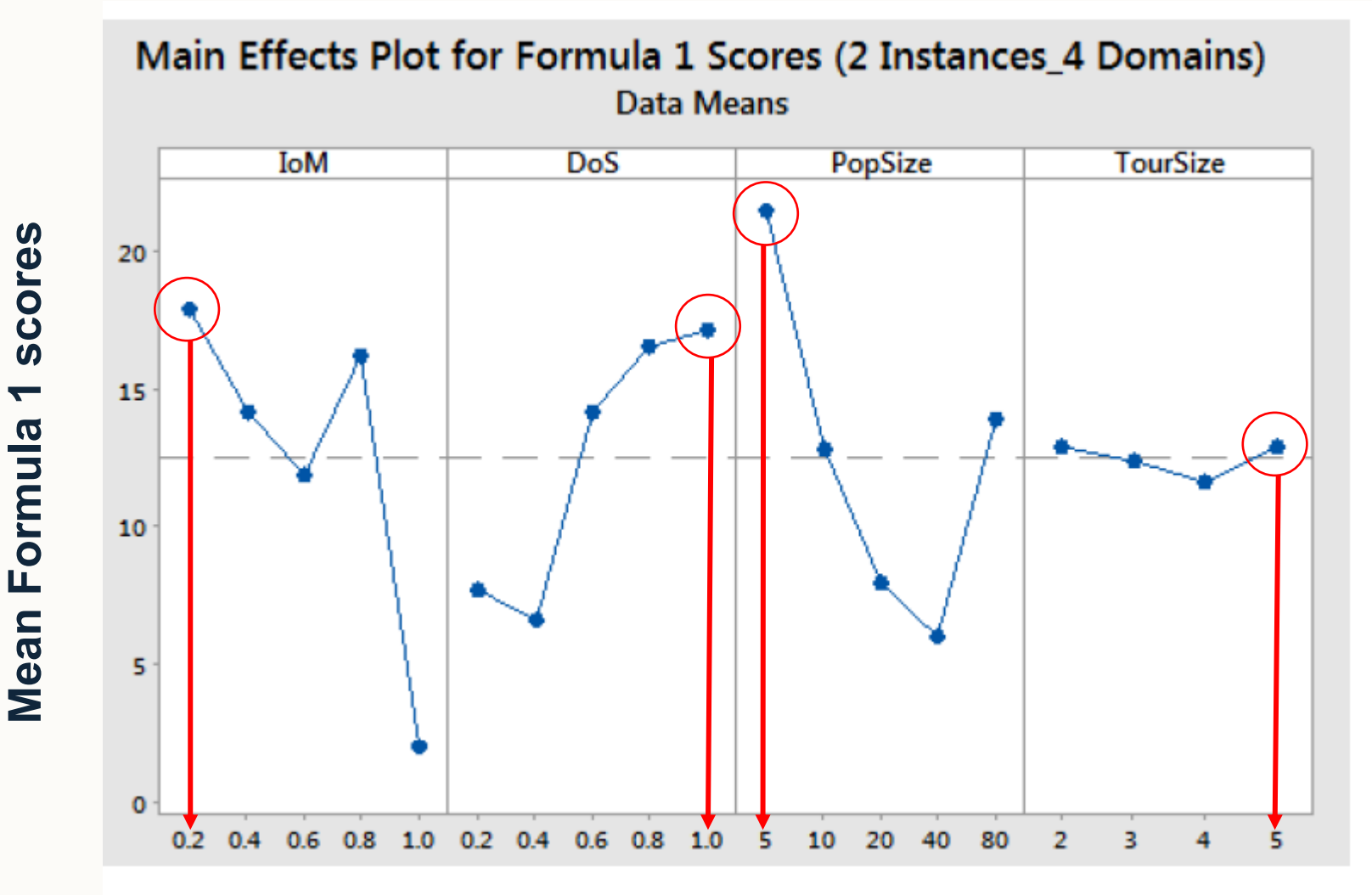
[1] E. Ozcan, S. Asta and C. Altintas, Memetic Algorithms for Cross-domain Heuristic Search, The 13th Annual Workshop on Computational Intelligence (UKCI), 2013, pp. 175-182. [Link to the paper]

Experimental Design

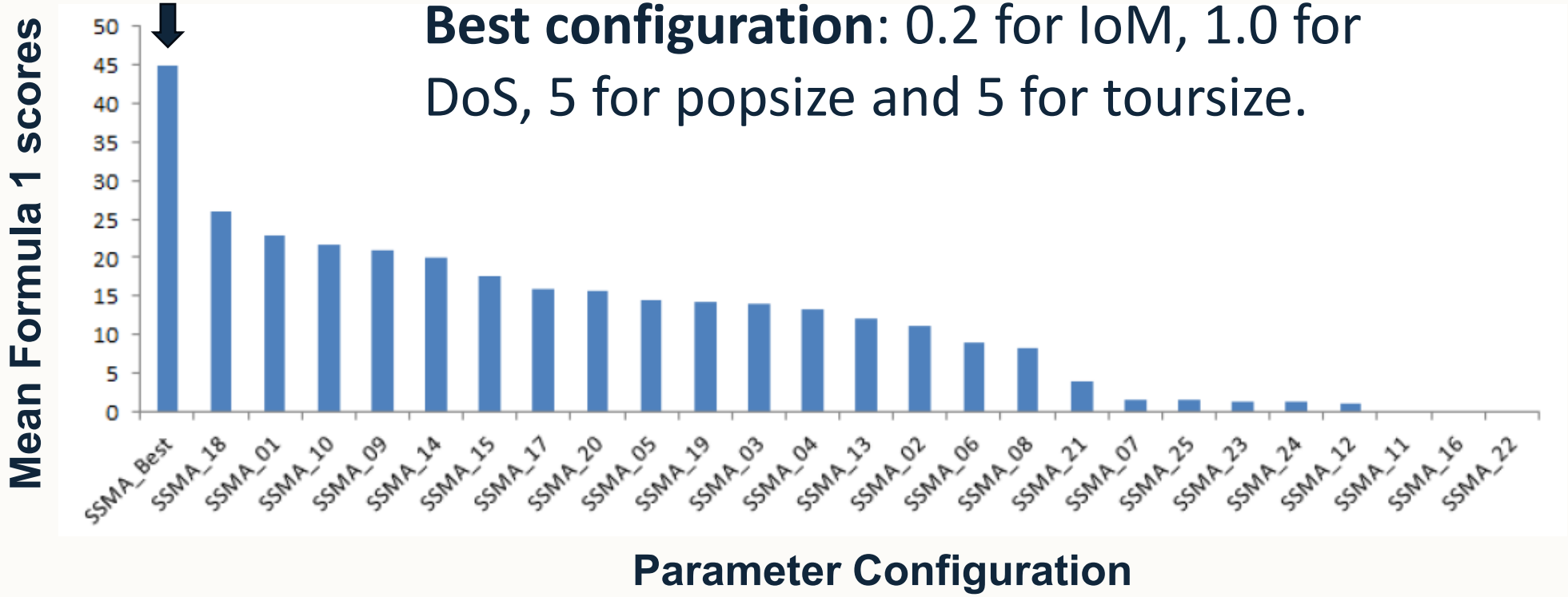
- **Training set** consists of 2 instances each from 4 “public” HyFlex problem domains.

- **L25 Taguchi Orthogonal Array** used to tune four parameters:
  - Intensity of mutation (IOM) $\in \{0.2, 0.4, 0.6, 0.8, 1.0\}$
  - Depth of search (DOS) $\in \{0.2, 0.4, 0.6, 0.8, 1.0\}$
  - Population size (PopSize) $\in \{5, 10, 20, 40, 80\}$
  - Tournament size (TourSize) $\in \{2, 3, 4, 5\}$

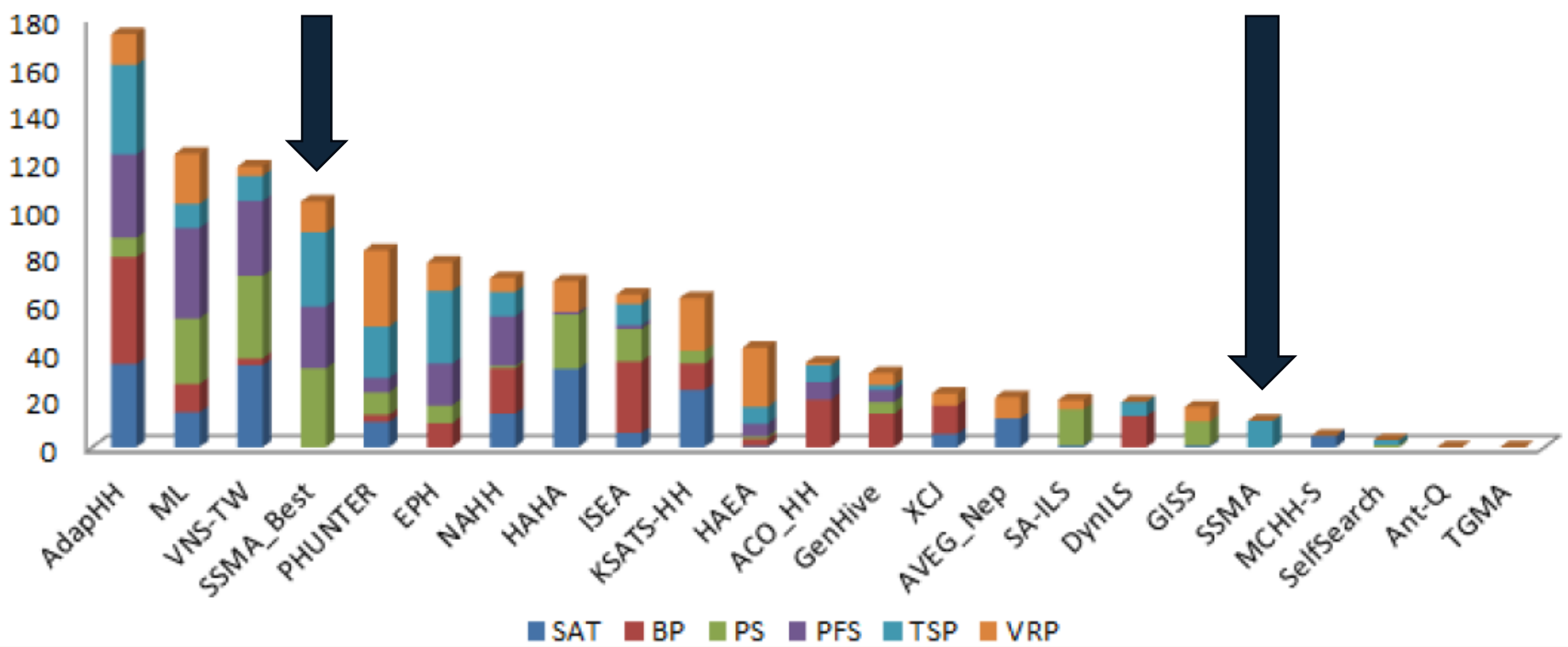
Main Effects Plot



Confirmation Run on Training Instances



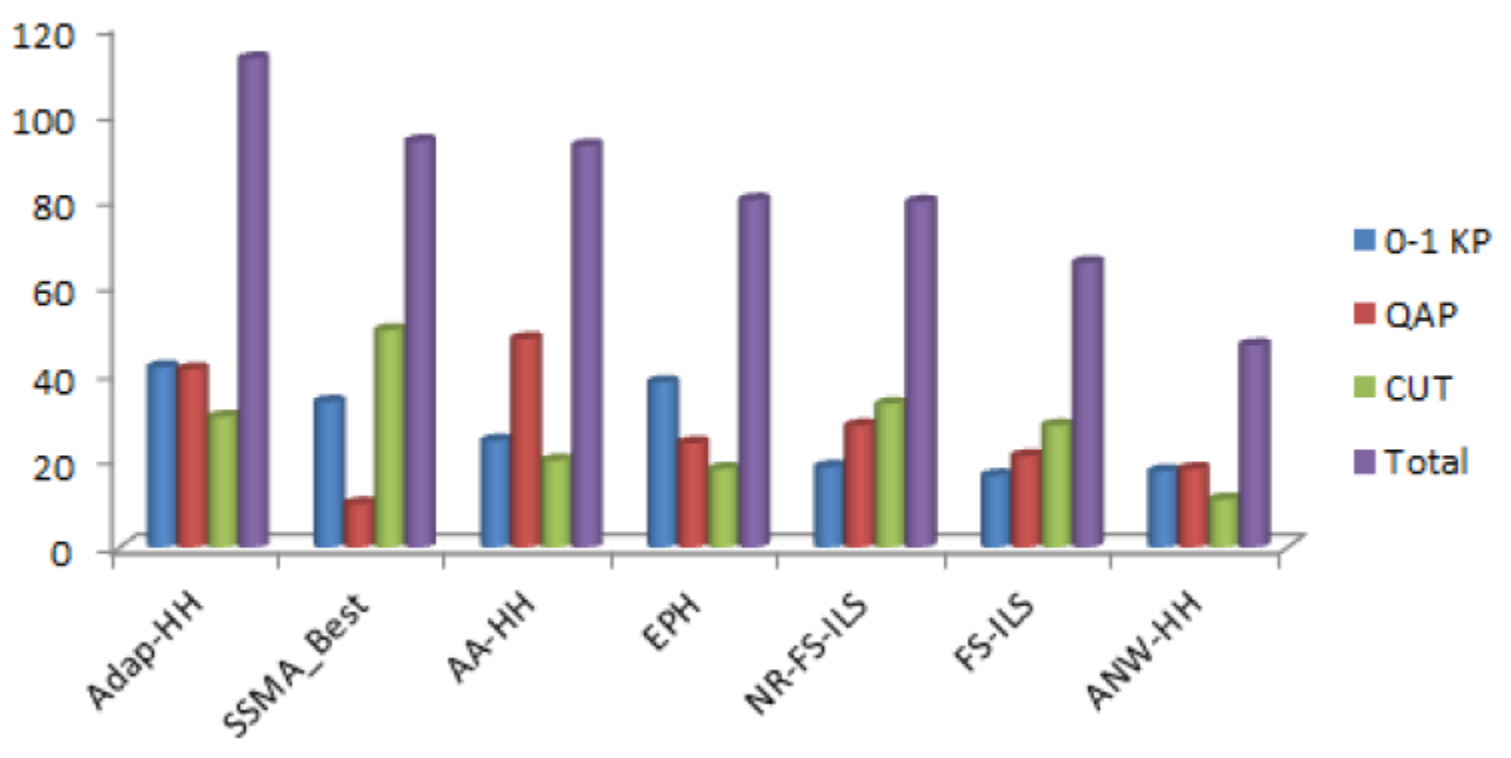
Cross-domain Performance of SSMA Compared to CHeSC 2011 Competitors



Effects of Cross-domain Parameter Tuning

		SSMA				SSMA-Best		
PD	ID	mean	median	best	vs.	mean	median	best
SAT	1	21.161	22	8	<	**9.484**	**9**	**3**
	2	52.484	53	37	<	**30.065**	**36**	**9**
	3	35	37	10	<	**15.387**	**10**	**3**
	4	27.742	27	26	<	**13.613**	**13**	**9**
	5	19.194	18	14	<	**12.742**	**13**	**9**
BP	1	0.083	0.082	0.074	<	**0.063**	**0.063**	**0.058**
	2	0.015	0.013	0.011	<	**0.011**	**0.012**	0.008
	3	0.022	0.022	0.018	>	0.034	0.034	0.029
	4	0.1115	0.1112	0.1105	<	**0.1102**	**0.11**	**0.1098**
	5	0.043	0.041	0.036	>	0.061	0.06	0.054
PS	1	51.129	50	37	<	**21.548**	**21**	**16**
	2	72015.613	70477	52056	<	**9705.129**	**9677**	**9477**
	3	13203.71	11859	5581	<	**3211.452**	**3219**	**3146**
	4	2942.419	2655	1820	<	**1596.871**	**1589**	**1344**
	5	435.645	431	385	<	**318.065**	**315**	**290**
PFS	1	6257.806	6258	6231	<	**6249.581**	**6251**	**6219**
	2	26884	26884	26813	<	**26812.452**	**26811**	**26754**
	3	6351.871	6363	6318	<	**6336.387**	**6333**	**6303**
	4	11441.806	11441	11410	<	**11376.613**	**11375**	**11333**
	5	26699.226	26703	26626	<	**26632.548**	**26640**	**26515**
TSP	1	48227.747	**48194.92**	**48194.92**	≤	**48221.583**	**48194.92**	**48194.92**
	2	21155458.17	21160875.64	20969185.56	<	**20912006.397**	**20885233.01**	**20789116.98**
	3	6825.552	6825.663	6800.708	<	**6811.145**	**6811.518**	**6799.111**
	4	68123.369	68059.971	67423.655	<	**67029.810**	**67043.255**	**66518.735**
	5	53810.138	53748.537	52685.992	<	**53503.576**	**53457.422**	**52247.568**
VRP	1	**71768.053**	71480.081	67820.589	≤	71946.305	**70776.497**	**65967.938**
	2	14324.522	14411.658	13358.611	<	**13826.295**	**13384.024**	**13328.791**
	3	176206.081	177131.584	167704.512	<	**147512.686**	**148001.434**	**143921.208**
	4	21647.018	21675.195	20678.096	<	**21275.493**	**21648.051**	**20654.219**
	5	152642.04	152829.839	149032.551	<	**147372.783**	**147228.056**	**145266.409**

Comparison of SSMA to State-of-the-art Algorithm on the Extended Domains



Interesting Observation

- Best configuration of SSMA uses TourSize = PopSize = 5.
- Both parents are selected as the best in the population and hence crossover produces an offspring identical to the best.
- Only remaining components with any influence are mutation and local search.
- Steady-state MA replaces the worst solution in the population with that just found; hence the re-configured SSMA is the same as Iterated Local Search accepting non-worsening moves.

Concluding remarks

- **Reminder COMP2001:**
  - In-lab Exam this Friday 13th March – details are on Moodle.
  - Project details to be released Monday 16th March.

- Module activities:
  - Formative self-assessment quiz on Moodle.
  - Next lecture: 17th March 4pm – Hyper-heuristics (Part 2).

University of Nottingham
UK | CHINA | MALAYSIA

Thank you

Any questions?